RISC-V

System-on-Chip Design

Harris, Stine, Thompson, & Harris

Chapter 11: MMU

Chapter 11 :: Topics

MMU: Memory Management Unit

- Principles
- RISC-V Practices
- Wally Implementation

Principles: MMU

- The MMU is responsible for handling:
  - **Virtual memory**
    - **TLB: translation lookaside buffer**
      - Cache of recently used page table entries
    - **HPTW:** hardware page table walker
      - efficiently fetches page table entries
  - **Memory protection**
    - **PMA:** physical memory attributes checker
      - checks address ranges and where to send accesses (physical memory, peripheral, etc.)
    - **PMP:** physical memory protection checker
      - Limits access type/privilege to certain memory regions.

Chapter 11: MMU

Virtual Memory

Virtual Memory Principles

- **Virtual memory provides:**
  - Illusion of a large memory (on disk) while only keeping a subset of data in physical memory
  - Ability to run concurrent processes
    - Protection between processes: processes don't access each other's memory

Virtual Memory Logistics

- **Processors generate virtual addresses**
  - Each process can use entire virtual address space, which provides illusion of a large, private, contiguous memory
- **Virtual memory (located on drive):** slow but large
- **Physical memory (RAM):** fast but slower
- Physical memory acts as **cache** for virtual memory:
  - A subset of virtual addresses are kept in physical memory
  - Page: chunk of memory moved from virtual to physical memory, typically 4 KiB
- The **MMU** (a hardware block in the processor) **translates virtual addresses** to physical addresses
- The **OS** moves pages from virtual to physical memory

Pages

- **Page size:** $P = 2^p$ bytes
- **Virtual address:** n bits ($N = 2^n$ bytes of virt mem)
  - Typically, n = 32/64 for 32/64-bit systems
  - Upper bits in 64-bit systems may not be used (few applications yet require $N = 2^{64}$ = 16 exabytes of memory)
- **Physical address:** m bits ($M = 2^m$ bytes of phys mem)

Thus,

- **Number of virtual pages:** 2^{n-p}
- **Number of physical pages:** 2^{m-p}

Number of pages

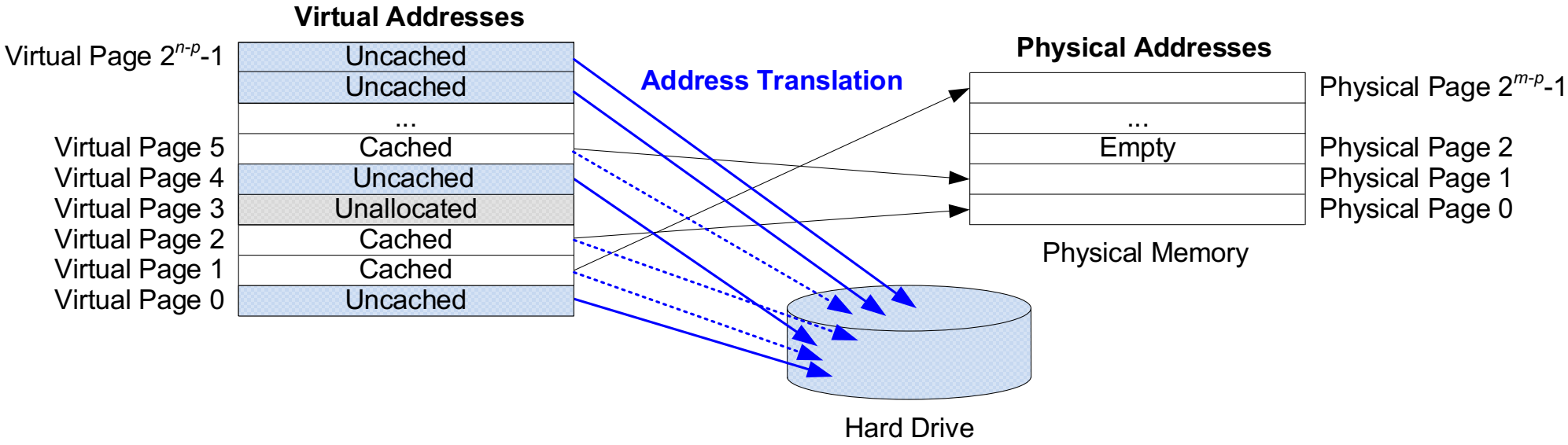
Example:

- **Page size:**
  - P = 4 KiB (2^p bytes), so
  - p = 12 bits (# bits needed to address each byte in page)
- **Physical memory (DRAM) size:**
  - M = 16 GiB (2^{34} bytes), so
  - m = 34 bits (# address bits)
- **Number of physical pages held in DRAM:** $2^{34-12} = 2^{22}$

Virtual Page Allocation

- **Unallocated:** No data associated with page
- **Cached:** An allocated page is in physical memory
- **Uncached:** An allocated page is on disk

Example allocations



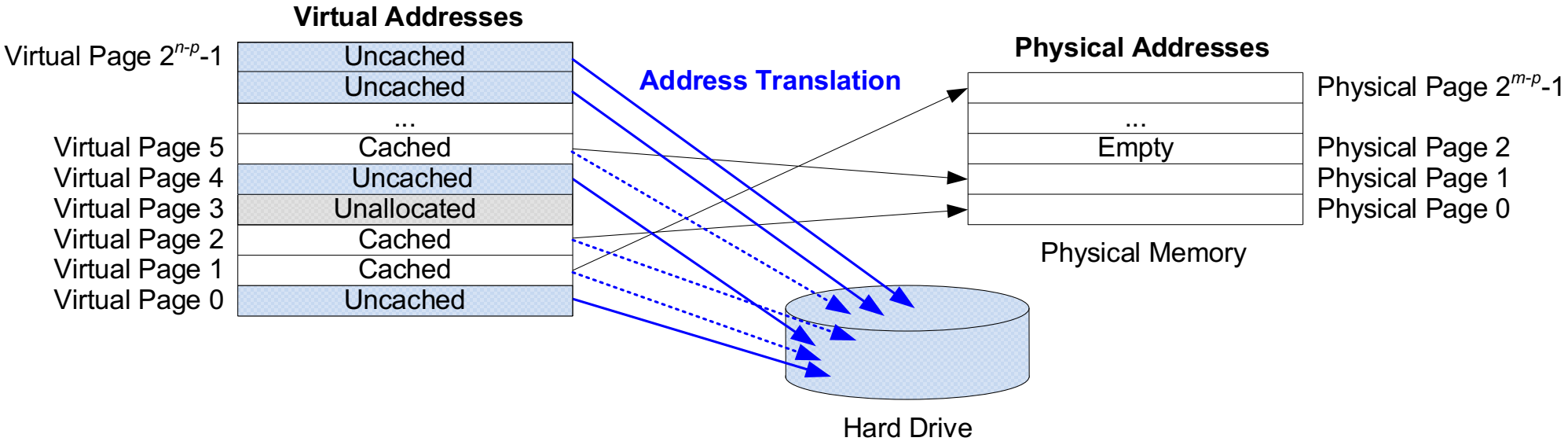
Chapter 11: MMU

Page Tables

Page Table

- **Page table:** Lists mapping of virtual to physical pages
  - It also holds the virtual page's status and the PPN (physical page number)
  - Each VPN (virtual page number) has an entry

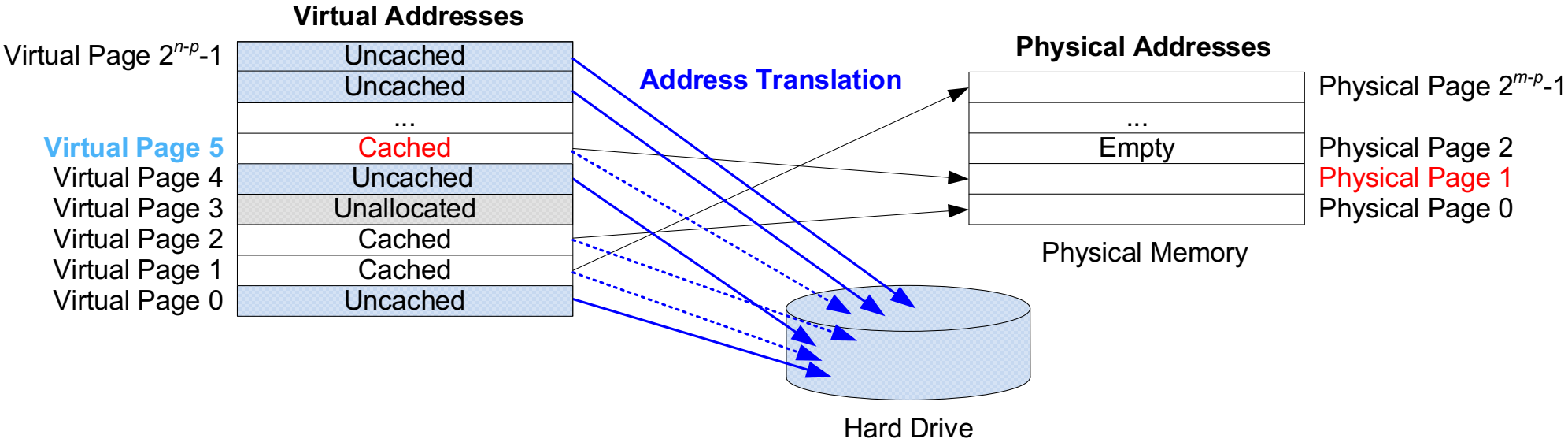
VPN	PPN	Status
2^{n-p}- 1		Uncached
2^{n-p}- 2		Uncached
...	...	...
5	000001	Cached
4		Uncached
3		Unallocated
2	000000	Cached
1	3FFFFF	Cached
0		Uncached



Page Table

- **Page table:** Lists mapping of virtual to physical pages
  - It also holds the virtual page's status and the PPN (physical page number)
  - Each VPN (virtual page number) has an entry

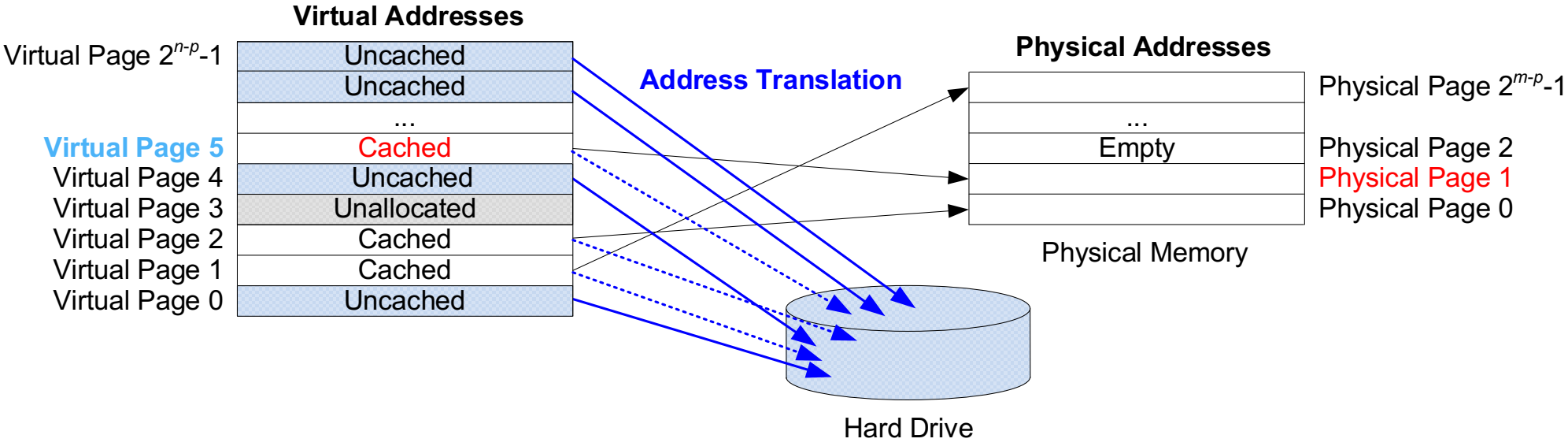
VPN	PPN	Status
2^{n-p}- 1		Uncached
2^{n-p}- 2		Uncached
...	...	...
5	000001	Cached
4		Uncached
3		Unallocated
2	000000	Cached
1	3FFFFF	Cached
0		Uncached



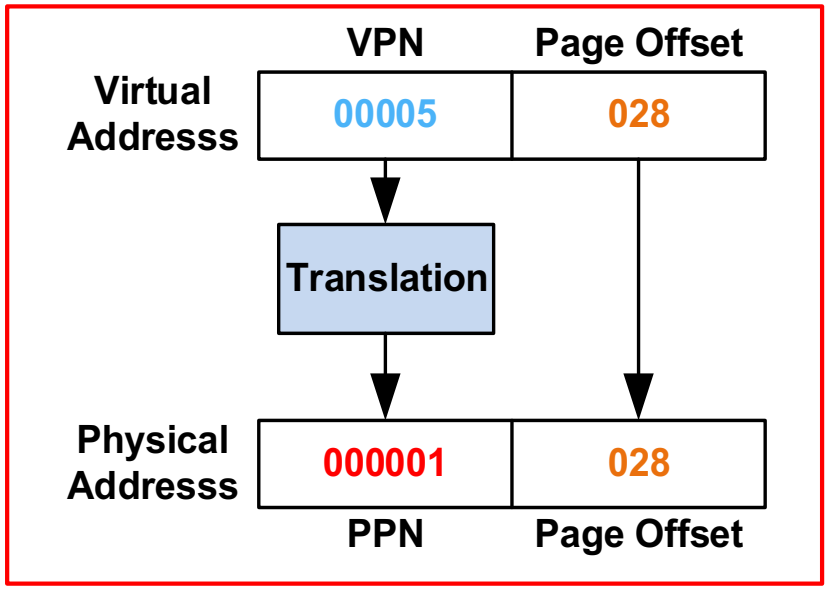
Page Table

- **Page table:** Lists mapping of virtual to physical pages
  - It also holds the virtual page's status and the PPN (physical page number)
  - Each VPN (virtual page number) has an entry

VPN	PPN	Status
2^{n-p}- 1		Uncached
2^{n-p}- 2		Uncached
...	...	...
5	000001	Cached
4		Uncached
3		Unallocated
2	000000	Cached
1	3FFFFF	Cached
0		Uncached

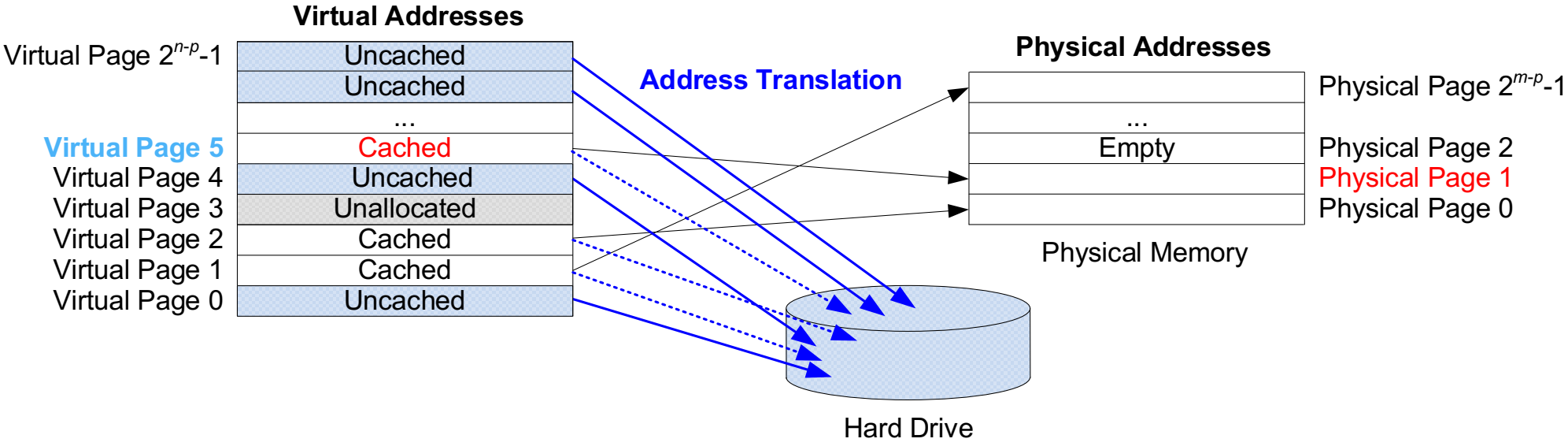


Address Translation



Page offset remains the same – only the page number is translated.

VPN	PPN	Status
2^{n-p}- 1		Uncached
2^{n-p}- 2		Uncached
...	...	...
5	000001	Cached
4		Uncached
3		Unallocated
2	000000	Cached
1	3FFFFF	Cached
0		Uncached



Page faults & Protection

- **Page fault:**
  - Occurs when a process accesses an uncached page
  - Processor takes a page fault exception
  - OS handles page fault by reading desired virtual page from disk and updating the page table entry.
- **Protection:**
  - Each page table entry (PTE) holds:
    - Status, PPN
    - Additional bits: indicate whether it is readable, writeable, executable, which protection level, etc.
  - Each process has its own page table, so they can't access each other's pages

Chapter 11: MMU

Multilevel Page Tables

Multilevel Page Tables

- Page table can be unreasonably large and be mostly unused (many PTEs are often unallocated)

Example 1: Determine page table size for:

- n = 32 (virtual address size), p = 12 (4 KiB pages)
- # PTEs = $2^{32-12} = 2^{20}$
- At 4 bytes/PTE, page table size is:

  2^{20} x 4 bytes = 2^{22} = **4 MiB**

Example 2: Determine page table size for:

- n = 64, p = 12 (4 KiB pages)
- # PTEs = $2^{64-12} = 2^{52}$
- At 8 bytes/PTE, page table size is:

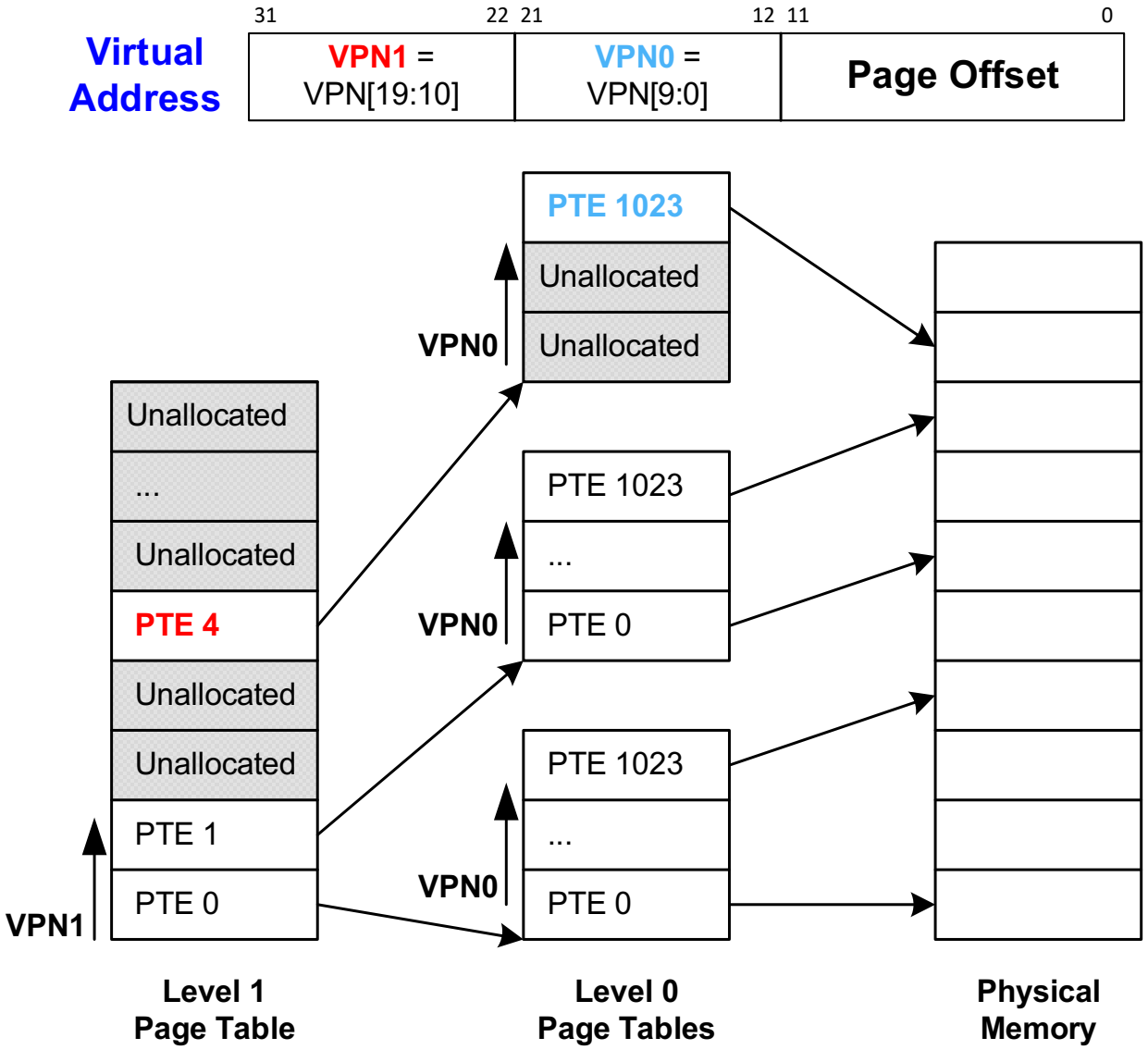
  2^{52} x 8 bytes = 2^{53} = **32 PiB**

Multilevel Page Table Hierarchy

- **Level 0 page tables:** hold PTE (page table entry)
- **Higher level page tables:** hold address of lower-level page tables

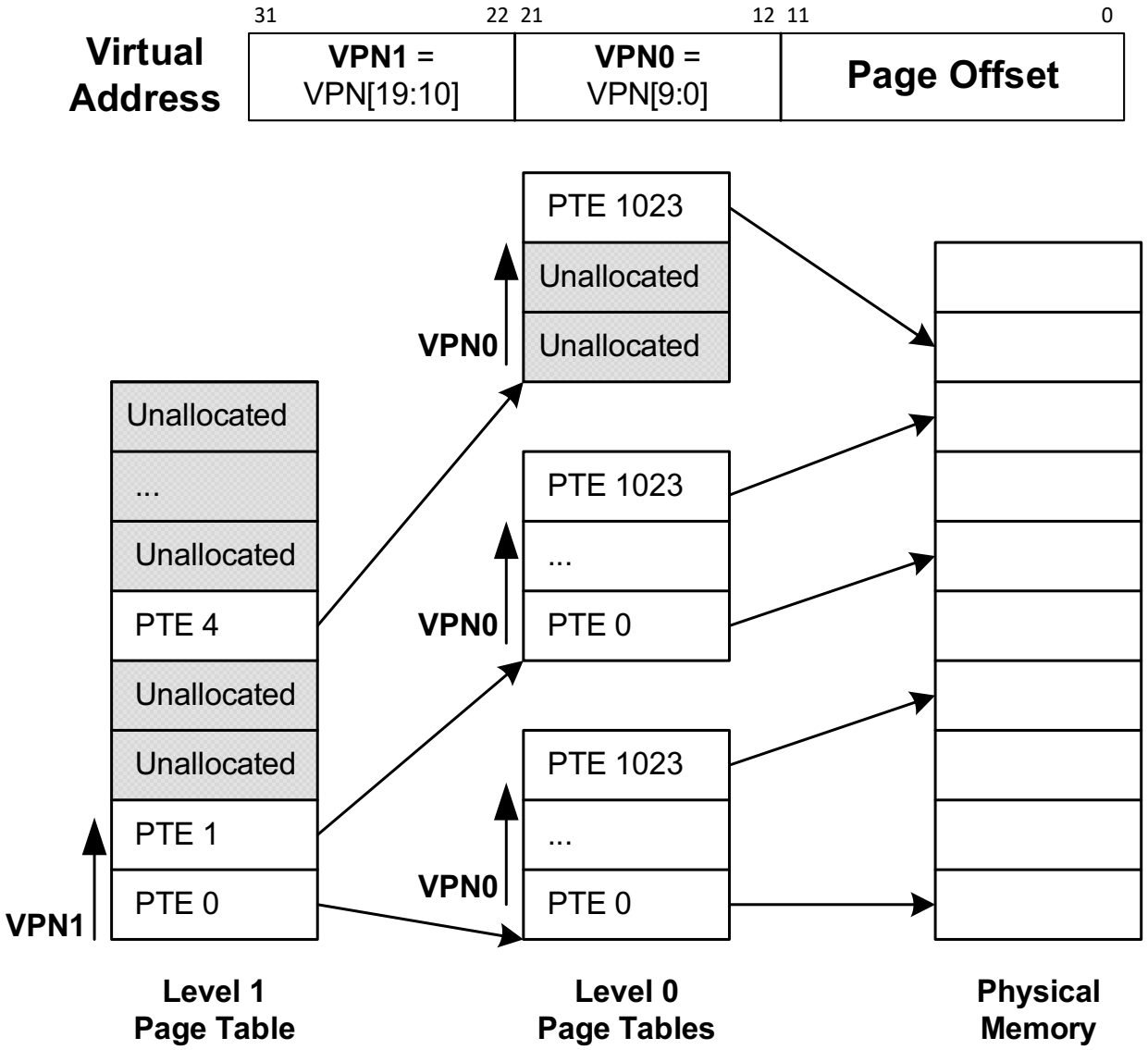
- **Virtual page numbers:**
  - **VPN0:** gives index into Level 0 page table
  - **VPN1:** gives index into Level 1 page table
  - Etc.

Multilevel Page Table Example



- **VPN1:** index into Level 1 page table
  - **Example: PTE 4** gives base address of Level 0 page table.
- **VPN0:** index into Level 0 page table
  - **Example: PTE 1023** gives PPN of a cached page

Multilevel Page Table Size



Individual Page Table Size:

- Each page table has 2^{10} entries (10-bit VPNs)
- Each PTE = 4 bytes
- So, each page table is 2^{10} x 4 bytes = 2^{12} bytes = 4 MiB

Page Tables:

- 1 Level 1 page table
- 3 allocated Level 0 page tables

Total Page Table Size:

- So, 4 page tables = 4 x 4 MiB = 16 MiB

Chapter 11: MMU

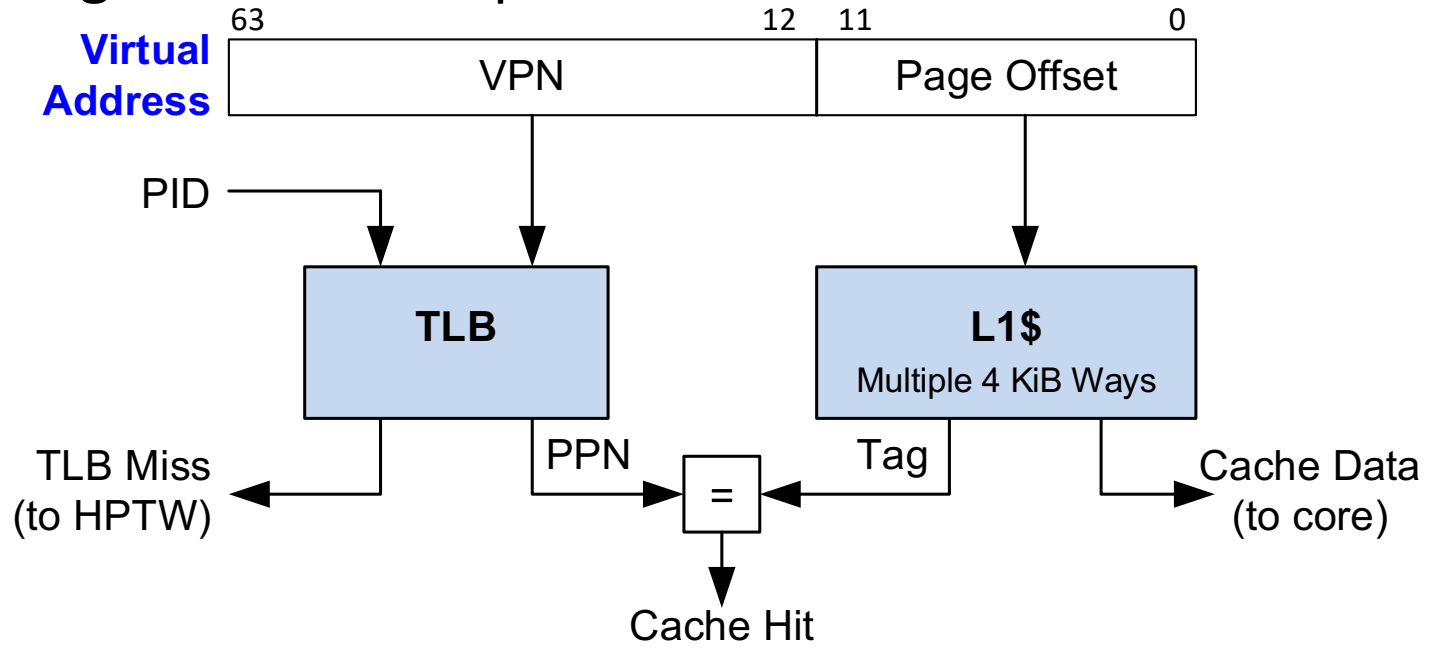
TLB

TLB: Translation Lookaside Buffer

- With virtual memory, each memory access takes at least **2 memory accesses:**
  - 1 to read page table (or more if multilevel)
  - 1 to access physical memory
- **Instead use TLB:**
  - Caches recent translations
  - High hit rate (99%+)
  - So, most memory accesses only take 1 access

TLB: Translation Lookaside Buffer

- Most systems have a D$ and I$: usually physically addressed
  - Set is given by page offset bits (lower bits of address)
  - TLB & cache access occurs simultaneously (using page offset bits)
  - Tag and PPN compared after these accesses occurs

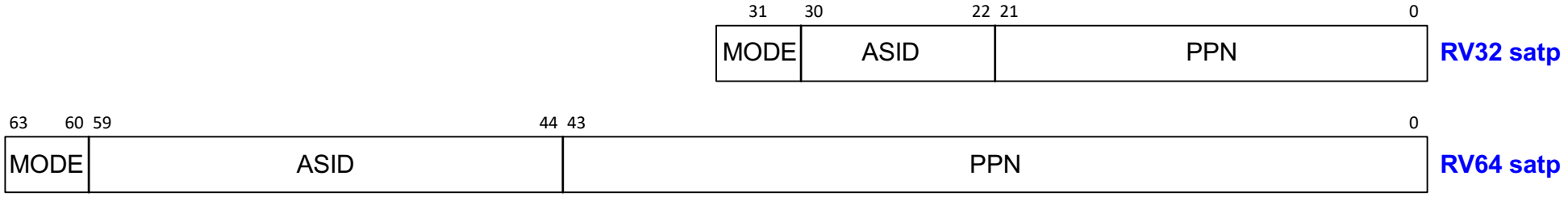


Chapter 11: MMU

RISC-V Practices

satp Register

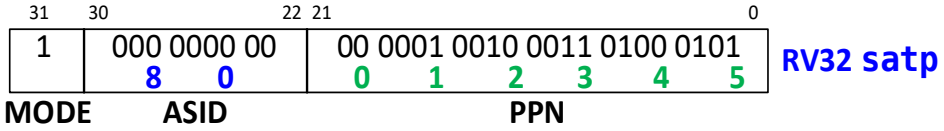
- **satp: Supervisor address translation and protection register**
  - Enables virtual memory
  - **PPN:** base address of the root (highest-level) page table: shift left by 12 bits to get physical address
  - **ASID:** address space identifier
    - Short PID (process ID) to differentiate multiple virtual address spaces in the TLB
  - **MODE:** bare metal (no virtual memory) or virtual memory mode (several exist for RV64). For RV32:
    - MODE = 0: no virtual memory (Bare mode)
    - MODE = 1: virtual memory

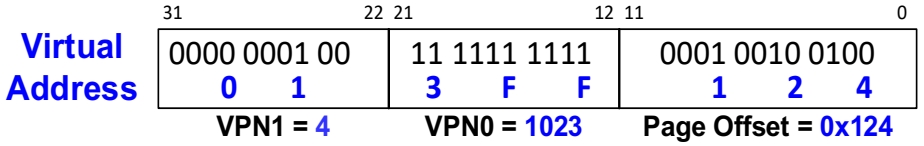


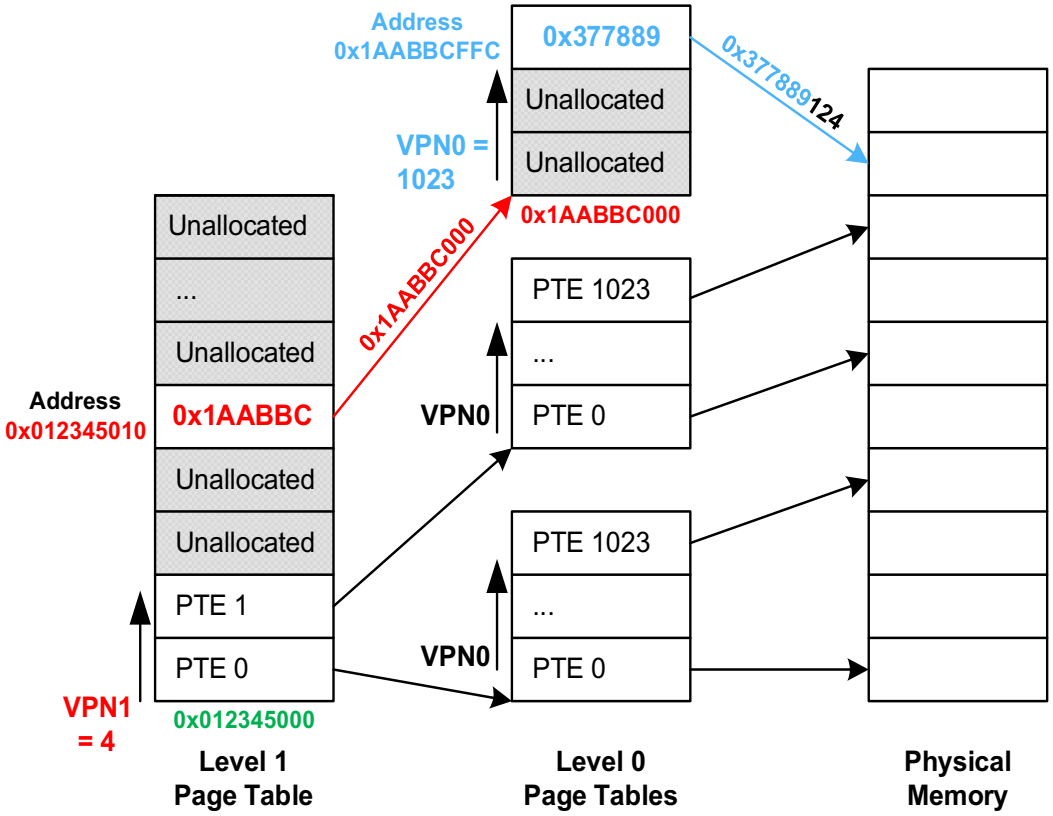
Chapter 11: MMU

Page Tables

Example 2-Level Page Table







satp = 0x8012345

- **MODE = 1** (virtual memory enabled)
- **PPN = 0x012345.** Address of Level 1 page table is 0x12345 << 12 = **0x012345000** (physical memory address is 34 bits)

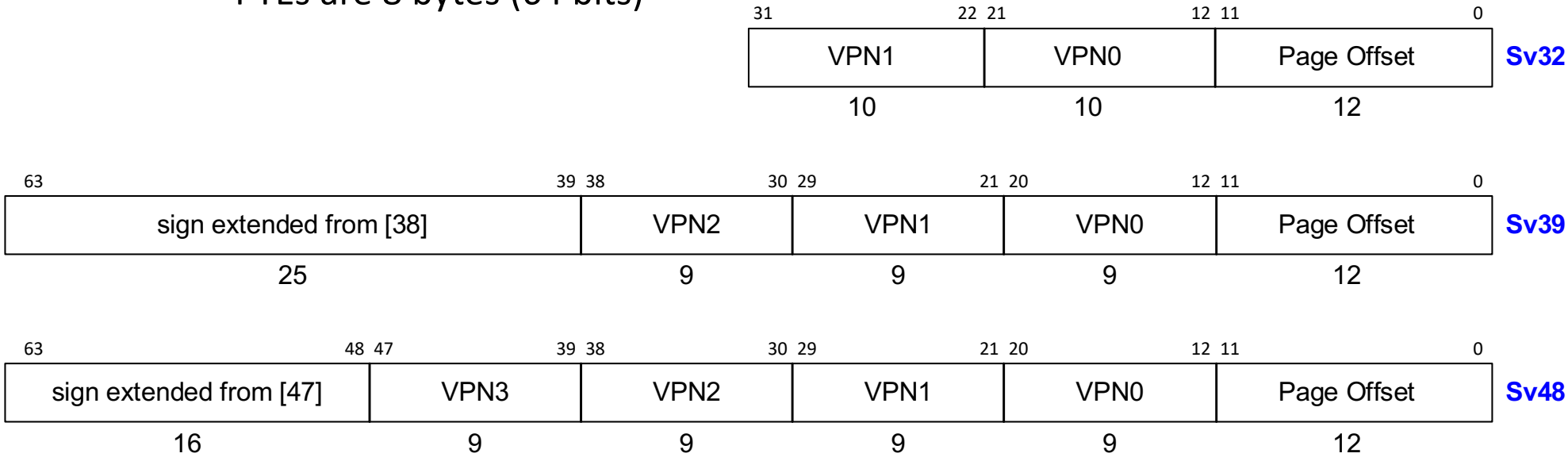
Virtual Address = 0x013FF124

- **VPN1 = 4:** entry 4 of Level 1 page table = **0x1AABBC**, so Level 0 page table is at address: 0x1AABBC << 12 = **0x1AABBC000**
- **VPN0 = 1023:** entry 1023 of Level 0 page table = **0x377889**, so append Page Offset (**0x124**) to this to get physical address:

0x377889124

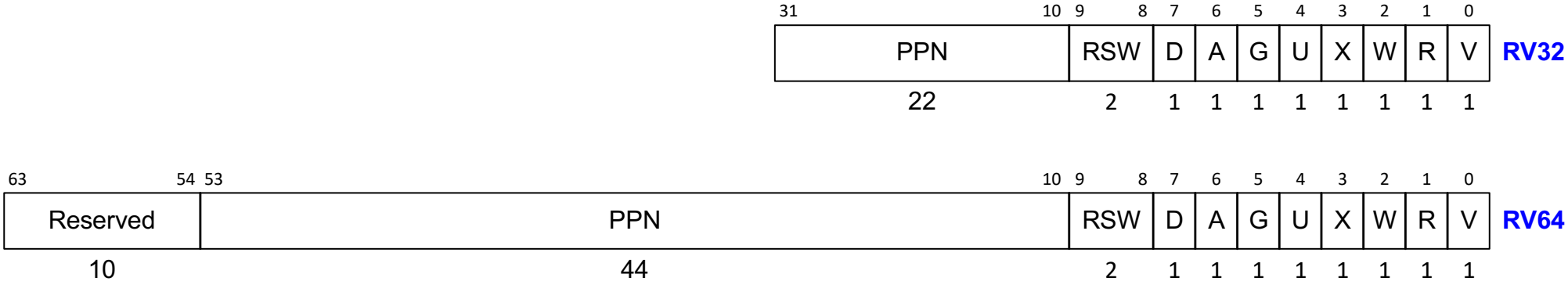
RISC-V Virtual Memory Modes (Sv*)

- **RV32:**
  - **Sv32:** virtual memory address = 32 bits
  - PTEs are 4 bytes (32 bits)
- **RV64:**
  - **Sv39:** virtual memory address = 39 bits (`satp` MODE = 1000)
  - **Sv48:** virtual memory address = 48 bits (`satp` MODE = 1001)
  - **Sv57:** virtual memory address = 57 bits (`satp` MODE = 1010)
  - PTEs are 8 bytes (64 bits)



PTEs: Page Table Entries

- **RV32:**
  - **PPN = 22 bits:** physical address = 22 + 12 = 34 bits (up to 2^{34} = 16 GiB)
- **RV64:**
  - **PPN = 44 bits:** physical address = 44 + 12 = 56 bits (up to 2^{56} = 64 PiB)



PTE Flags

Flag	Meaning
V	**Valid:** 1 if the page is in main memory, 0 if not
X, W, R	**Execute, Write, Read** Permissions: 000: Pointer to next level of page table 001: Read only 010: <illegal> 011: Read-write 100: Execute only 101: Read-execute 111: Read-write-execute
U	**User** mode
G	**Global:** page exists for all address spaces; ignore ASID in TLB matching
A	**Accessed:** page has been accessed at least once since A bits were last cleared
D	**Dirty:** at least one byte from the page has been written
RSW	**Reserved** for operating system

Variable Page Sizes: Superpages

Format	Leaf Page Table Level	Page Offset Bits	Page Size	Page Type
Sv32	0	12	4 KiB	kilopage (normal)
Sv32	1	12 + 10 = 22	4 MiB	**megapage**
Sv39	0	12	4 KiB	kilopage (normal)
Sv39	1	12 + 9 = 21	2 MiB	**megapage**
Sv39	2	12 + 9 × 2= 30	1 GiB	**gigapage**
Sv48	0	12	4 KiB	kilopage (normal)
Sv48	1	12 + 9 = 21	2 MiB	**megapage**
Sv48	2	12 + 9 × 2 = 30	1 GiB	**gigapage**
Sv48	3	12 + 9 × 3 = 39	512 GiB	**terapage**

Variable Page Sizes: Superpages

Format	Leaf Page Table Level	Page Offset Bits	Page Size	Page Type
Sv32	0	12	4 KiB	kilopage (normal)
Sv32	1	12 + 10 = 22	4 MiB	**megapage**
Sv39	0	12	4 KiB	kilopage (normal)
Sv39	1	12 + 9 = 21	2 MiB	**megapage**
Sv39	2	12 + 9 × 2= 30	1 GiB	**gigapage**
Sv48	0	12	4 KiB	kilopage (normal)
Sv48	1	12 + 9 = 21	2 MiB	**megapage**
Sv48	2	12 + 9 × 2 = 30	1 GiB	**gigapage**
Sv48	3	12 + 9 × 3 = 39	512 GiB	**terapage**

Each page table is **4 KiB** (2^12 B).

- **Sv32:** each PTE is 32 bits (4 bytes = 2^2 bytes), so:
  - Each page table has 2^12/2^2 = 2^10 entries.
  - **10 bits** are used as the VPN at each level.
- **Sv39 and Sv48:** each PTE is 64 bits (8 bytes = 2^3 bytes), so:
  - Each page table has 2^12/2^3 = 2^9 entries.
  - **9 bits** are used as the VPN at each level.

Chapter 11: MMU

PMA & PMP

Memory Attributes & Protection

- **PMA (physical memory attributes) checker:**
  - **Checks for:**
    - Where to direct access (memory, peripheral, etc.)
    - Cacheability
  - Produces **access fault** if unsuccessful

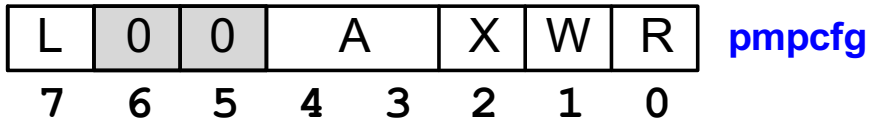
- **PMP (physical memory protection) checker:**
  - Enforces protected address ranges

PMA (Attributes) for RV64gc

Region	Base Address	Range	Access Width	XWR	Cacheable	Idempotent	AMO
BOOTROM	00001000	FFF	Any	XR	YES	YES	NO
SDC	00012100	1F	32/64-bit	RW	NO	NO	NO
CLINT	02000000	FFFF	Any	RW	NO	NO	NO
PLIC	0C000000	3FFFFFF	32-bit	RW	NO	NO	NO
UART0	10000000	07	8-bit	RW	NO	NO	NO
GPIO	10060000	FF	32-bit	RW	NO	NO	NO
UNCORE_ RAM	80000000	7FFFFFFF	Any	XWR	YES	YES	YES

PMP: Physical Memory Protection

- **By default, PMP:**
  - grants M-mode access to all memory
  - Denies S- and U-mode access to all memory
- **PMP Registers:**
  - `pmpaddr0`…`pmpaddr63`: 0, 16, or 64 PMP address registers:
  - `pmpconfig`: fields associated with each `pmpaddr` register
    - **XWR** always correspond to S- and U-mode, unless L = 1
    - **A:** type of region (see table)



A	Name	Region Type
00	OFF	Disabled
01	TOR	Top of Range
10	NA4	Naturally aligned 4-byte region
11	NAPOT	Naturally aligned power of two region (≥ 8 bytes)

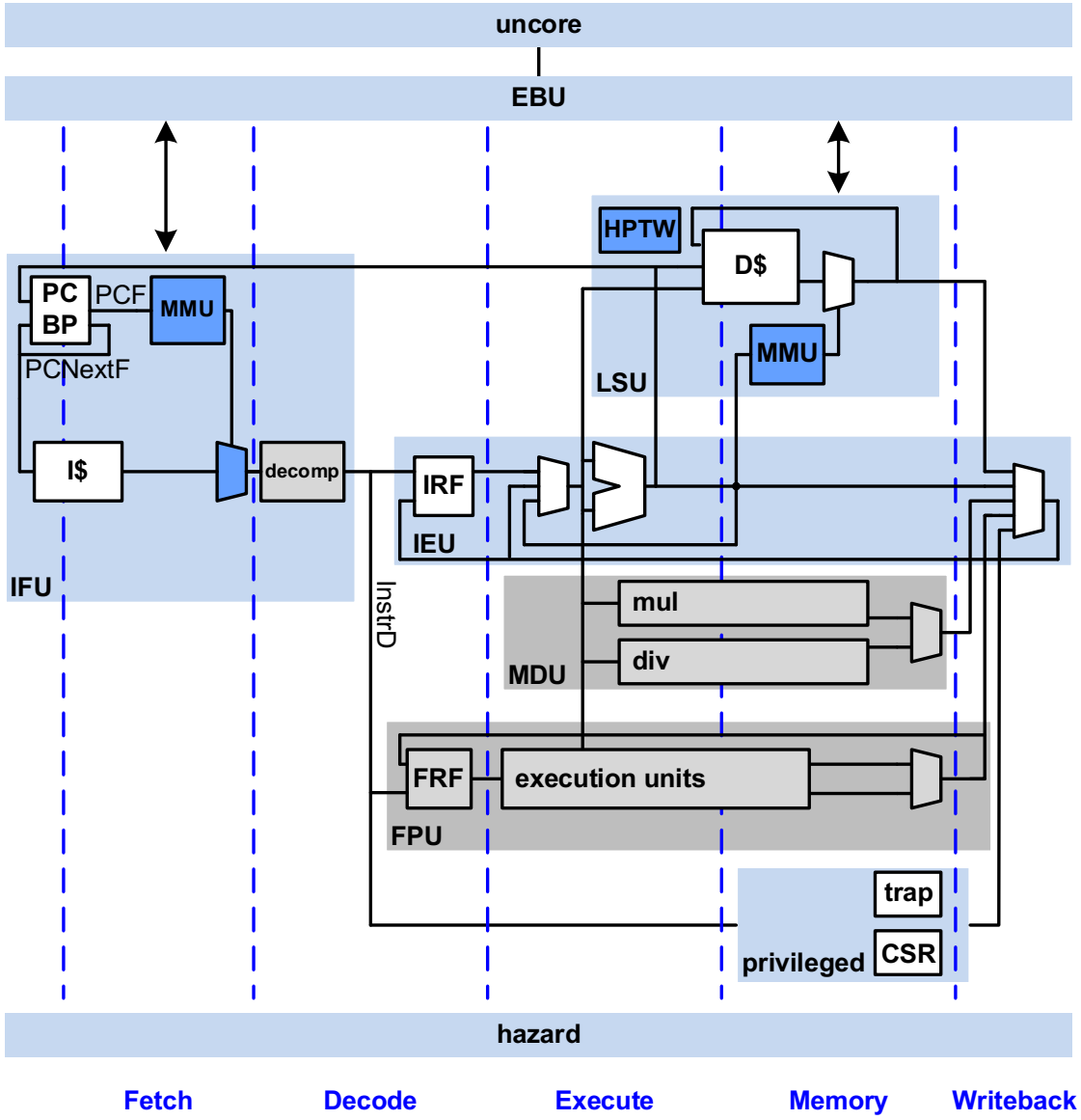
PMP Regions Example

region	pmpaddr	pmpcfg	L	A	X	W	R	Comments
0	0FFFFFFF	1F	0	NAPOT	0	1	1	I/O 00000000-7FFFFFFF RW
1	20040000	00	0	OFF	0	0	0	NULL
2	2004003F	09	0	TOR	0	0	1	80100000-801000FF R
3	20040080	00	0	OFF	0	0	0	NULL
4	20040084	0C	0	TOR	1	0	0	80100200-80100210 X
5	200400C0	90	1	NA4	0	0	0	80100300-80100303 locked out
6	2004013F	18	0	NAPOT	0	0	0	80100400-801005FF no access
7	00000000	00	0	OFF	0	0	0	NULL
8	00000000	00	0	OFF	0	0	0	NULL
9	00000000	00	0	OFF	0	0	0	NULL
10	00000000	00	0	OFF	0	0	0	NULL
11	00000000	00	0	OFF	0	0	0	NULL
12	00000000	00	0	OFF	0	0	0	NULL
13	00000000	00	0	OFF	0	0	0	NULL
14	00000000	00	0	OFF	0	0	0	NULL
15	2FFFFFFF	1F	0	NAPOT	1	1	1	Main mem 80000000-FFFFFFFF RWX

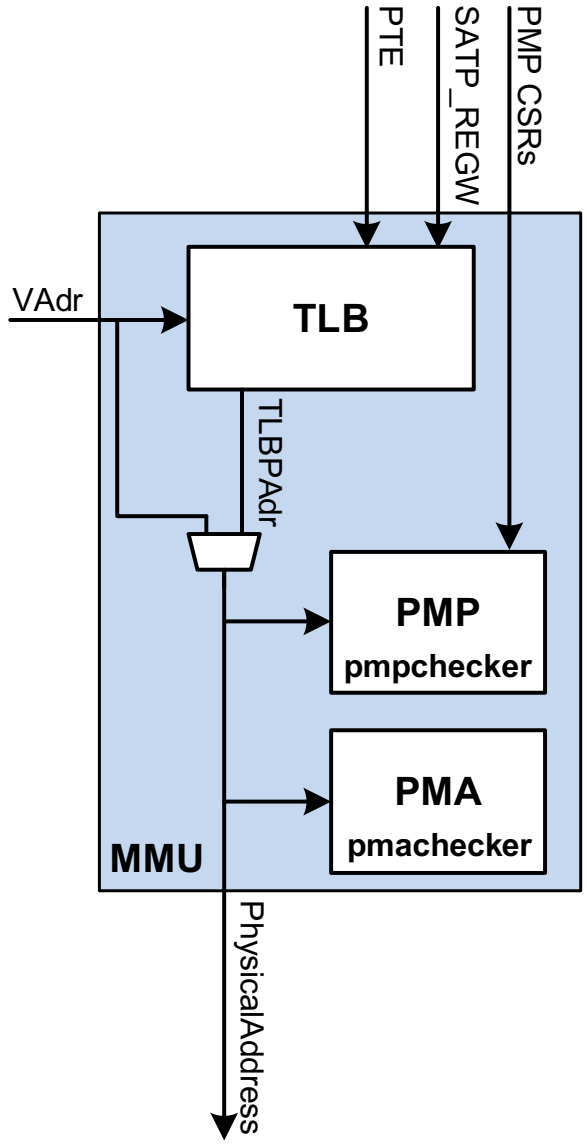
Chapter 11: MMU

Wally Implementation

Wally Top-Level Block Diagram



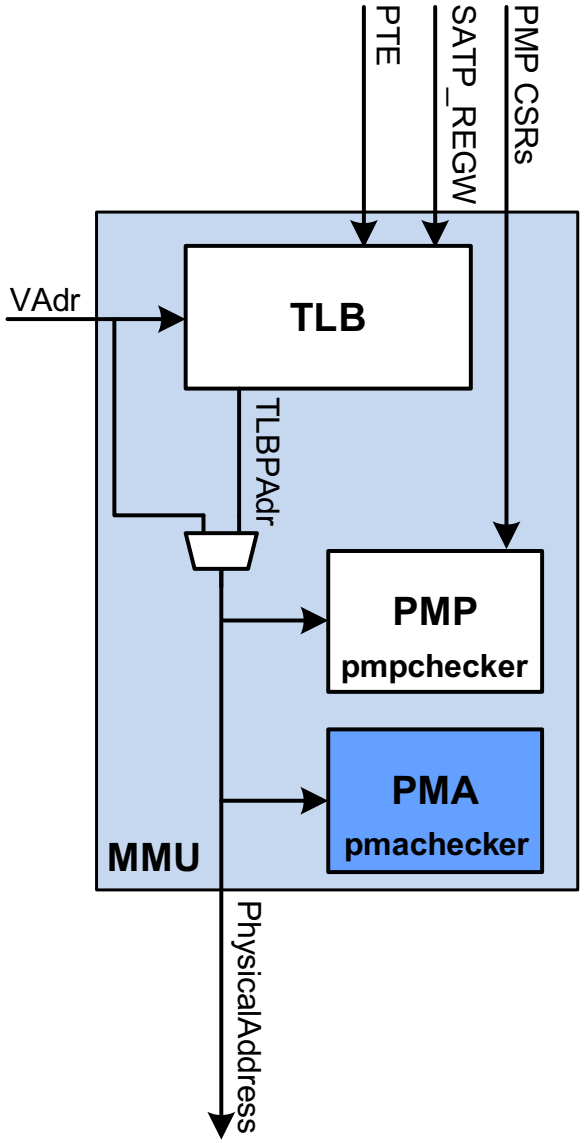
Wally MMU Top-Level Block Diagram



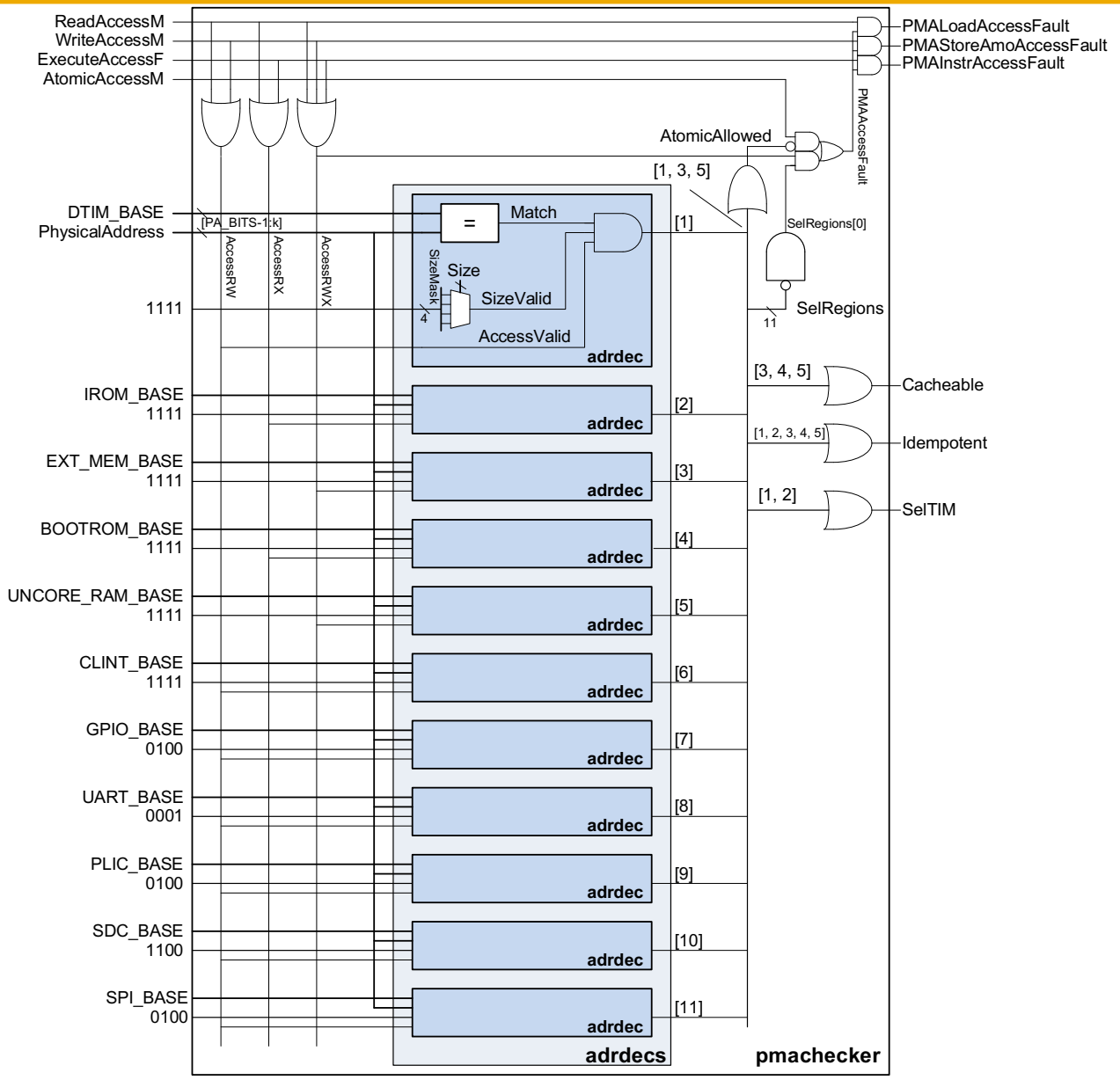
MMU:

- **Produces physical address (`PhysicalAddress`):**
  - If virtual memory, **translates** virtual address (`VAdr`) to physical address
  - If no virtual memory, `PhysicalAddress = VAdr`
- **PMA** checks regions and **PMP** checks privilege and access type – they generate faults if violated
- **On TLB miss**, MMU sends a request to **HPTW to fetch missing PTE**

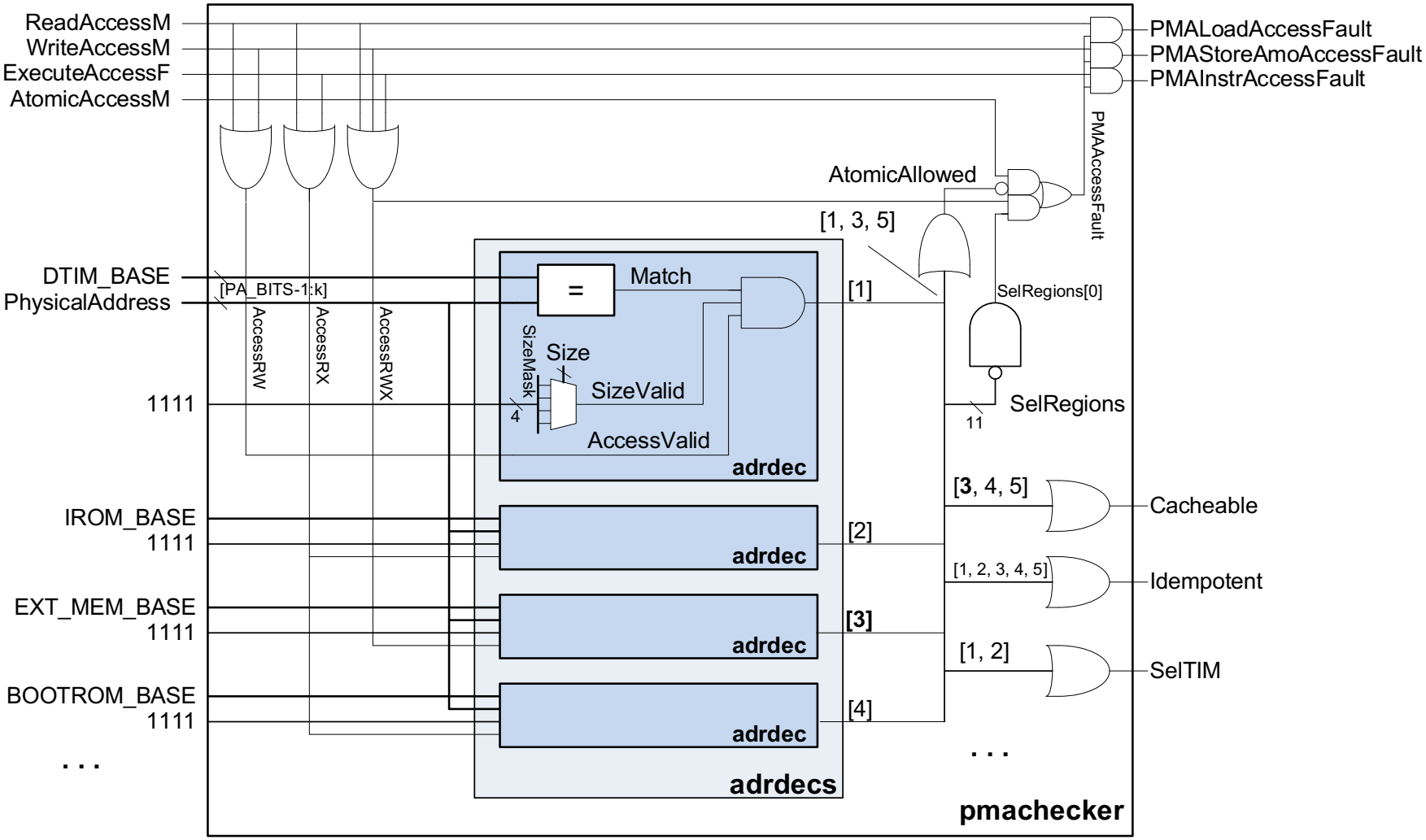
Wally PMA Checker



Wally PMA Checker

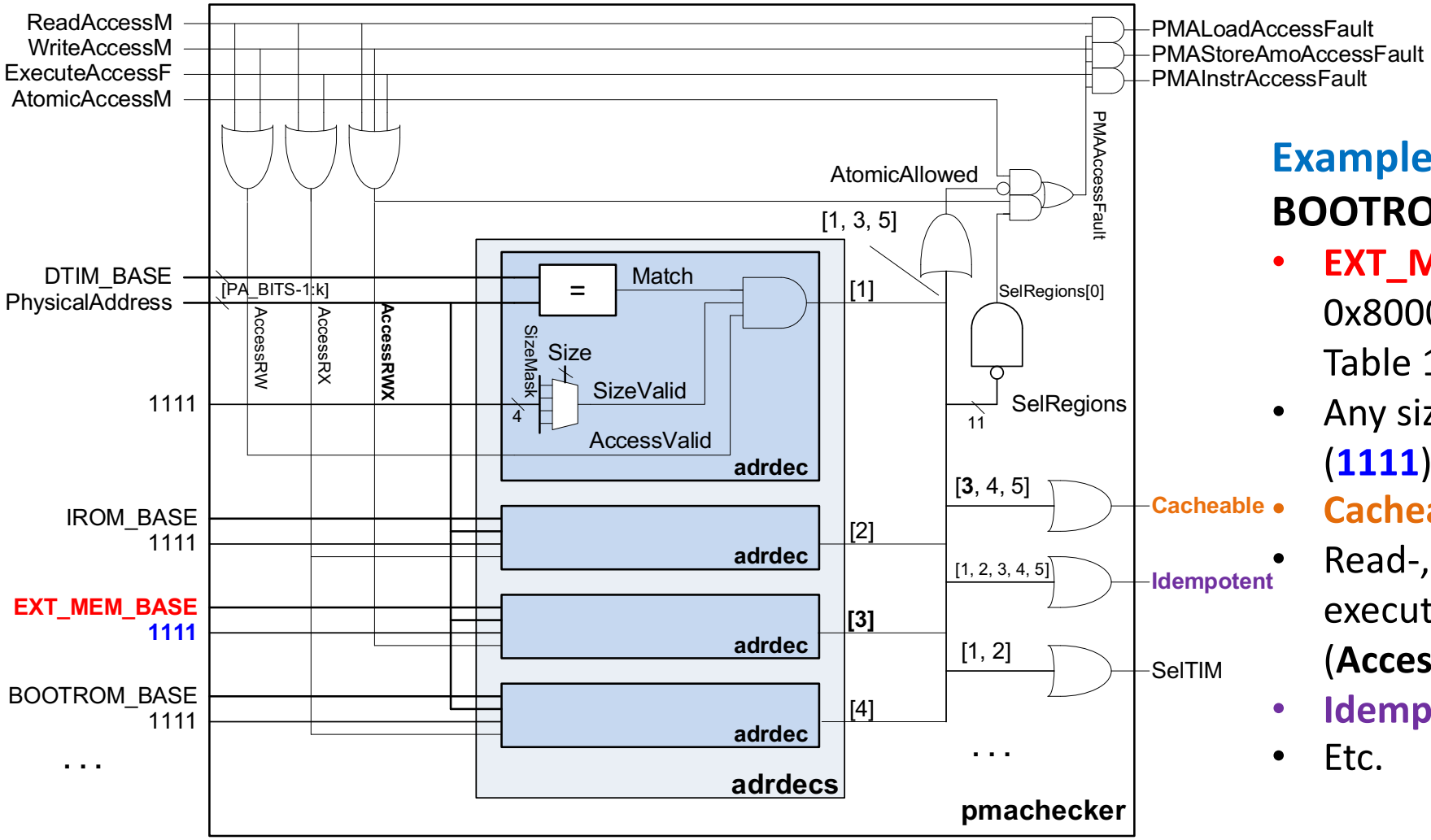


Wally PMA Checker (Partial)



PMA: Checks regions and attributes

Wally PMA Checker: Attributes

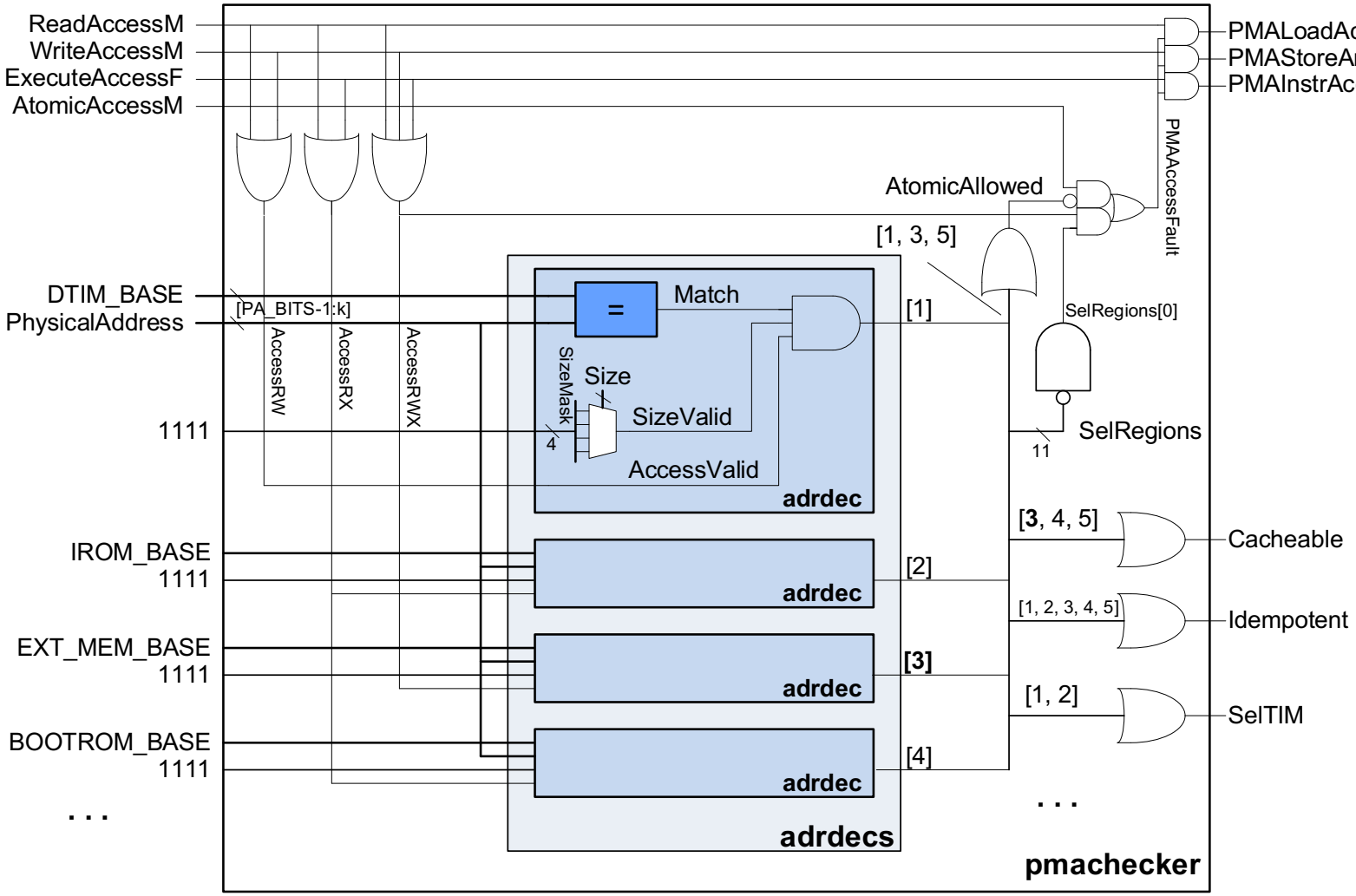


Example 1:
BOOTROM

- **EXT_MEM_BASE** = 0x8000_0000 (see Table 1.6)
- Any size access (**1111**)
- **Cacheable** (Sel[3])
- Read-, write-, or executable (**AccessRWX**)
- **Idempotent**
- Etc.

PMA: Checks regions and attributes

Wally PMA Checker: Region



Match =
(BASE == PA)

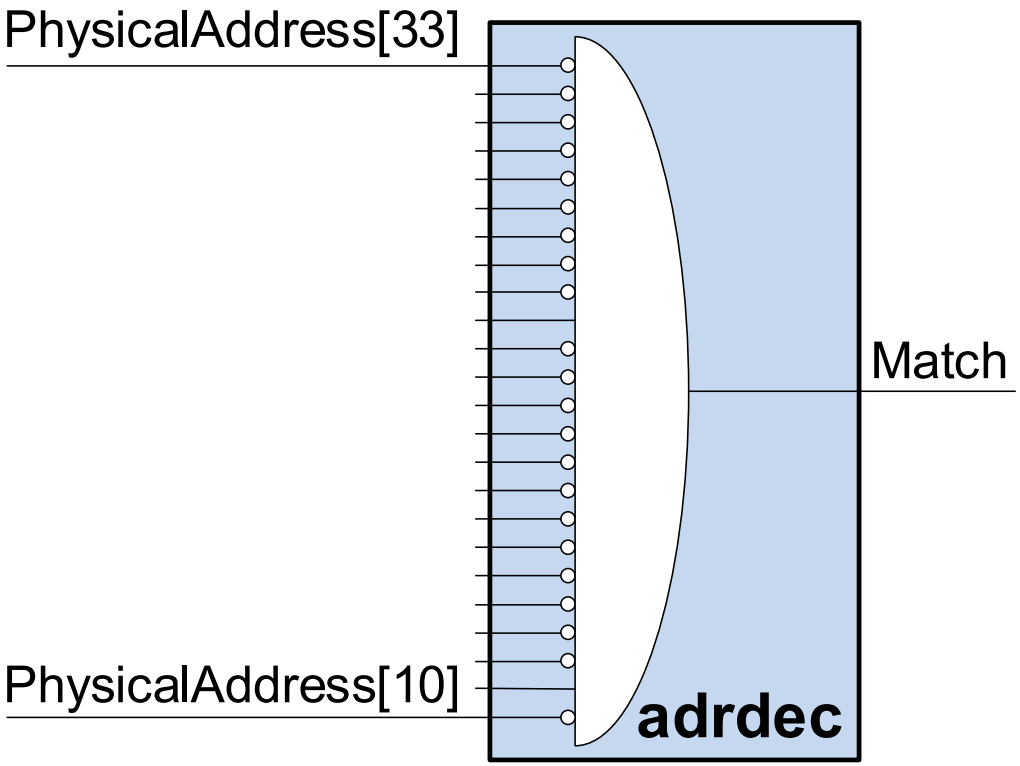
Matches on upper bits only (region).

PMA: Checks regions and attributes

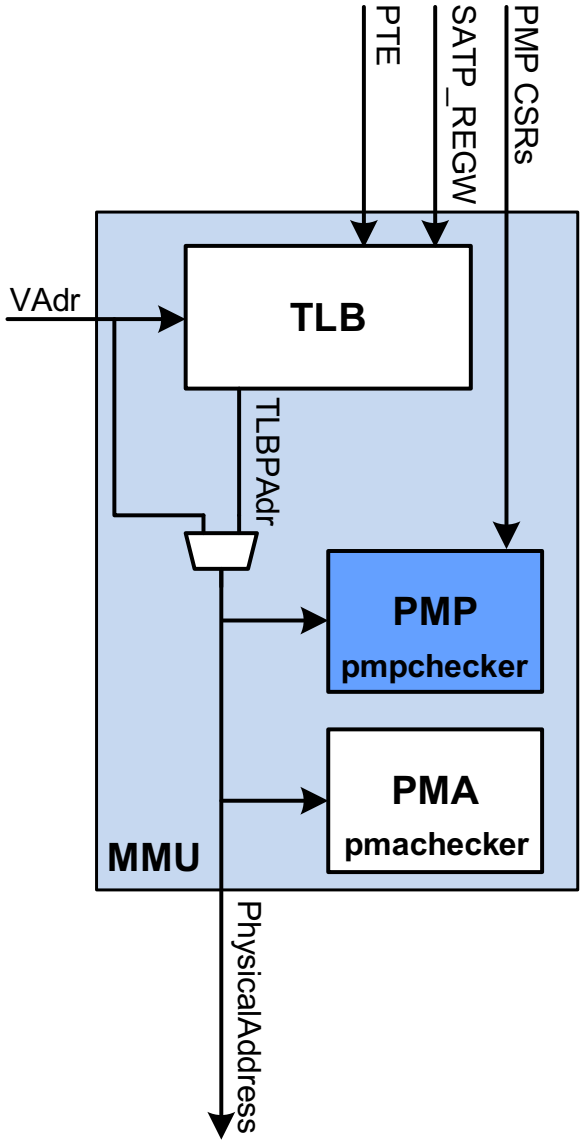
Match Logic

Example:

- Region spanning: 0x00100_0800 – 0x00100_0BFF
- Start of region: **00 0000 0001 0000 0000 0000 10**00 0000 0000
- End of region: **00 0000 0001 0000 0000 0000 10**11 1111 1111
- Top 24 bits ([33:10]) must be the same



Wally PMP Checker

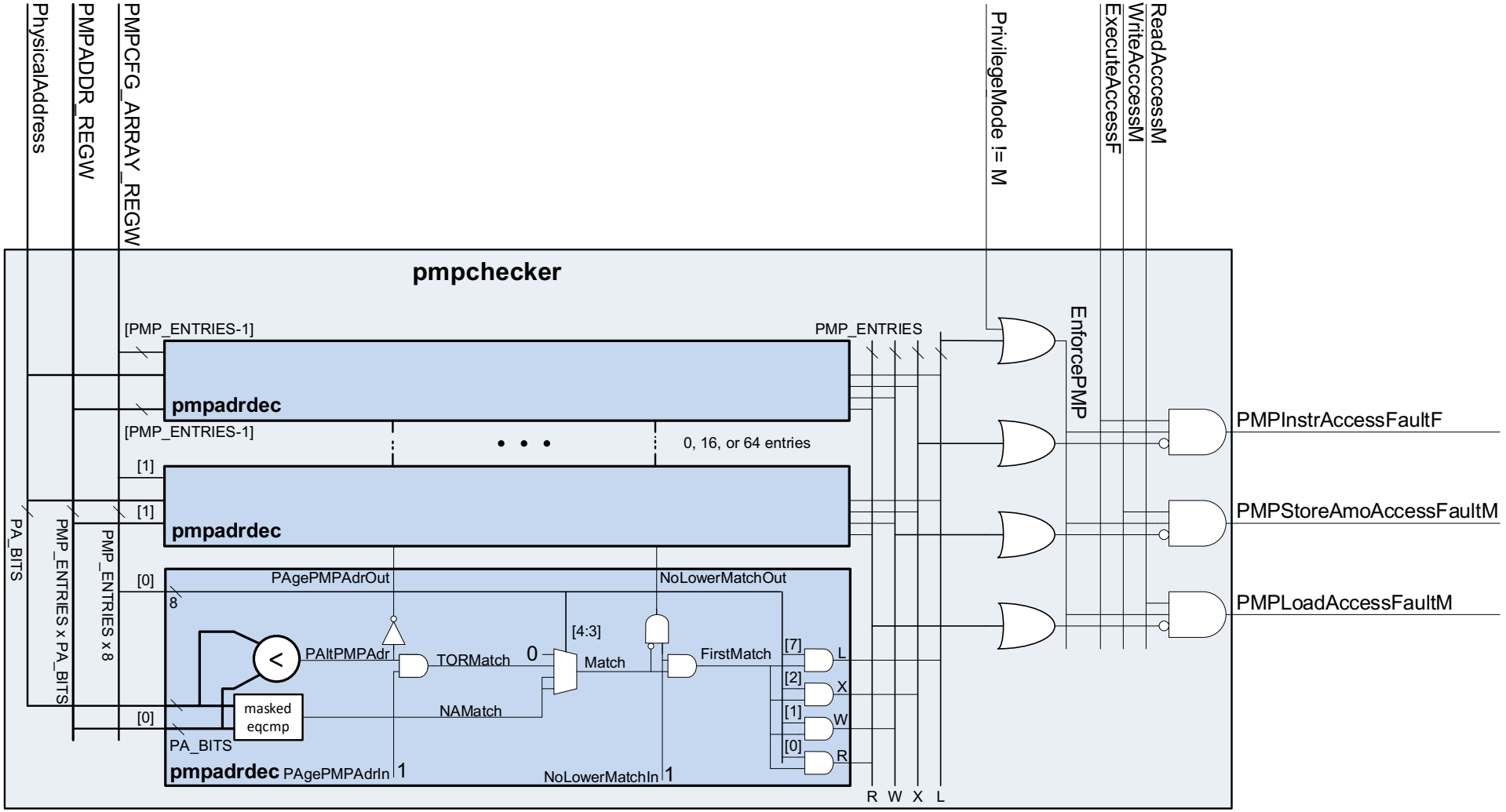


Wally PMP Checker

PMP (physical memory protection): Cause access fault if permissions violated. It contains:

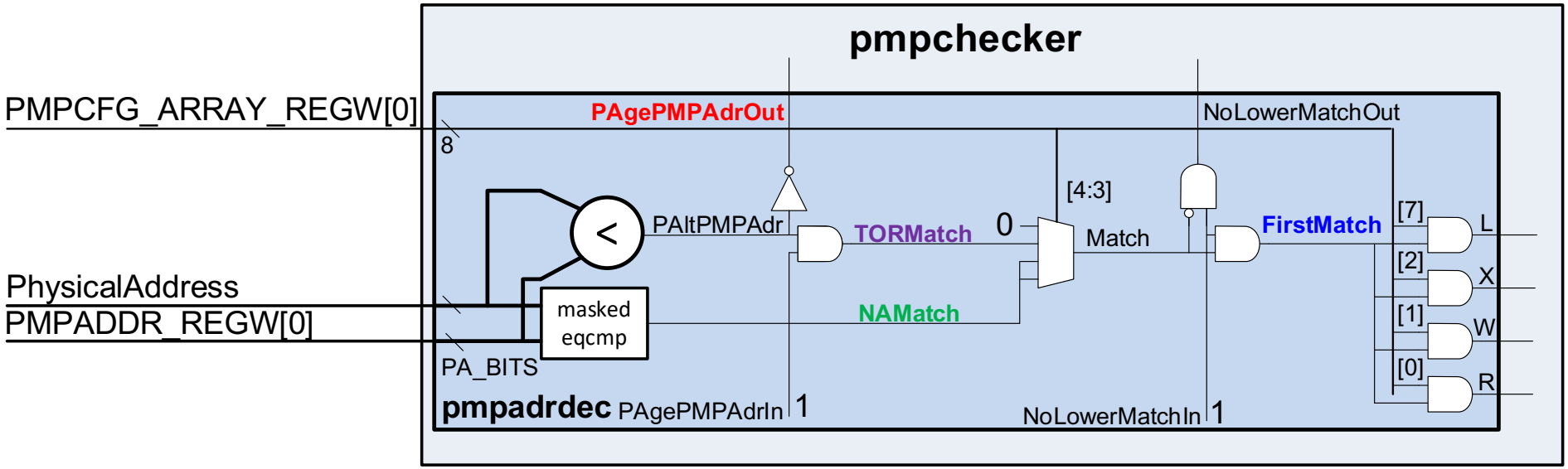
- **16-64 address decoders**
  - Checks if within range encoded in `pmpaddr0` (e.g., upper bits of `PhysicalAddress` < `PMPADDR_ARRAY_REGW[0]`)
  - `FirstMatch` = top PMP entry that matches
- **R, W, X, L extracted from `pmpcfg` to check if need to enforce rules**
  - If M-mode, only enforce if L (locked) bit = 1
  - If other modes, enforce RWX

Wally PMP Checker



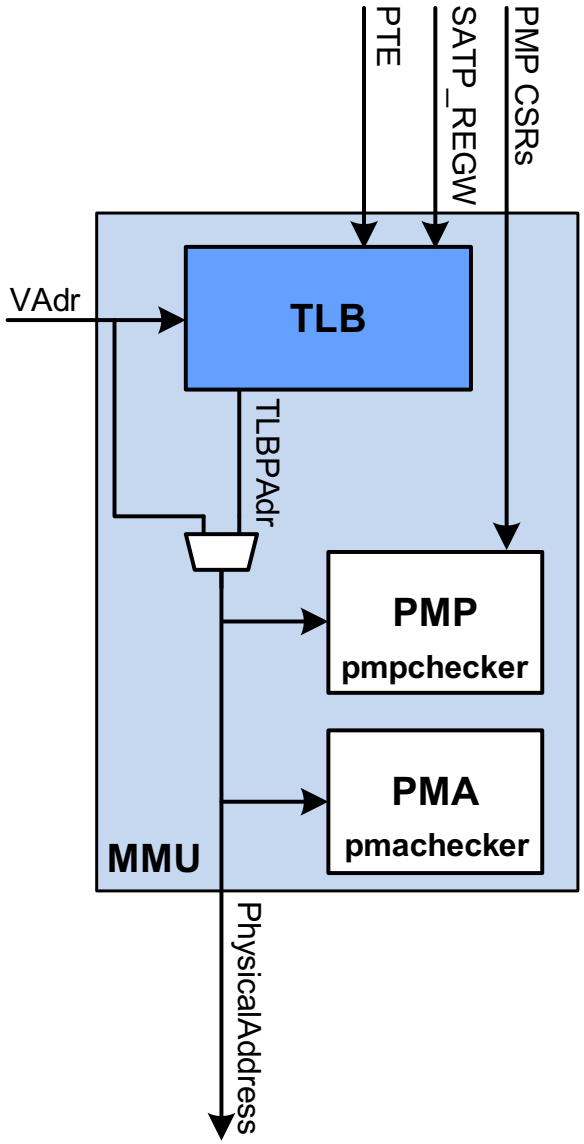
PMP: Causes access fault if permissions violated

Wally PMP Checker (Partial)



- **Address decoder:**
  - PAgePMPAdr (Physical address >= PMP address):
    - `PAgePMPAdr` = `~(PhysicalAddress < PMPADDR_ARRAY_REGW[0])`
  - **Masked eqcmp:** checks for top of range (**TOR**) and naturally aligned (**NA**) matches
  - `FirstMatch` = top PMP entry that matches

Wally MMU: TLB

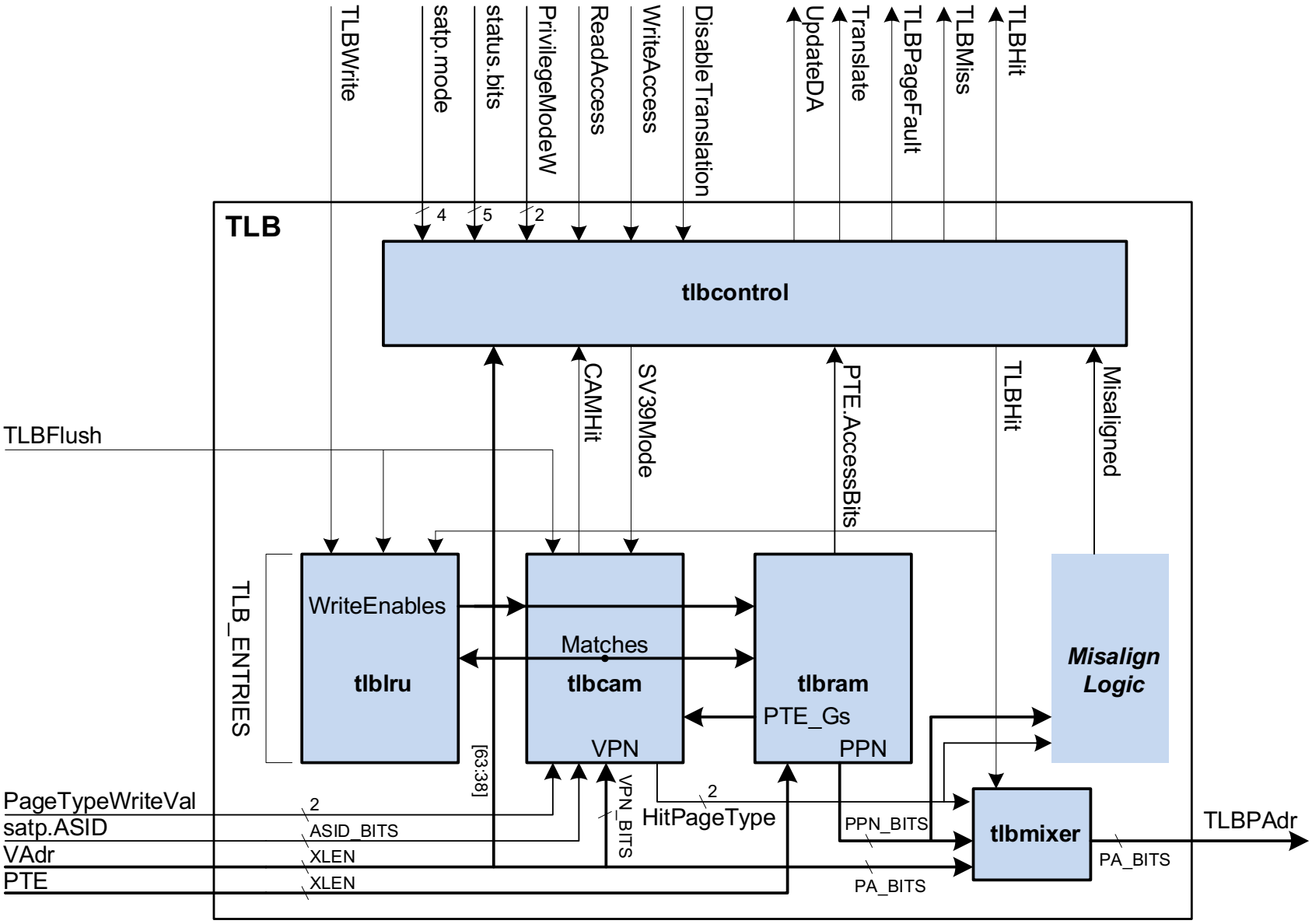


Wally TLB

TLB: fully associative cache of address translations

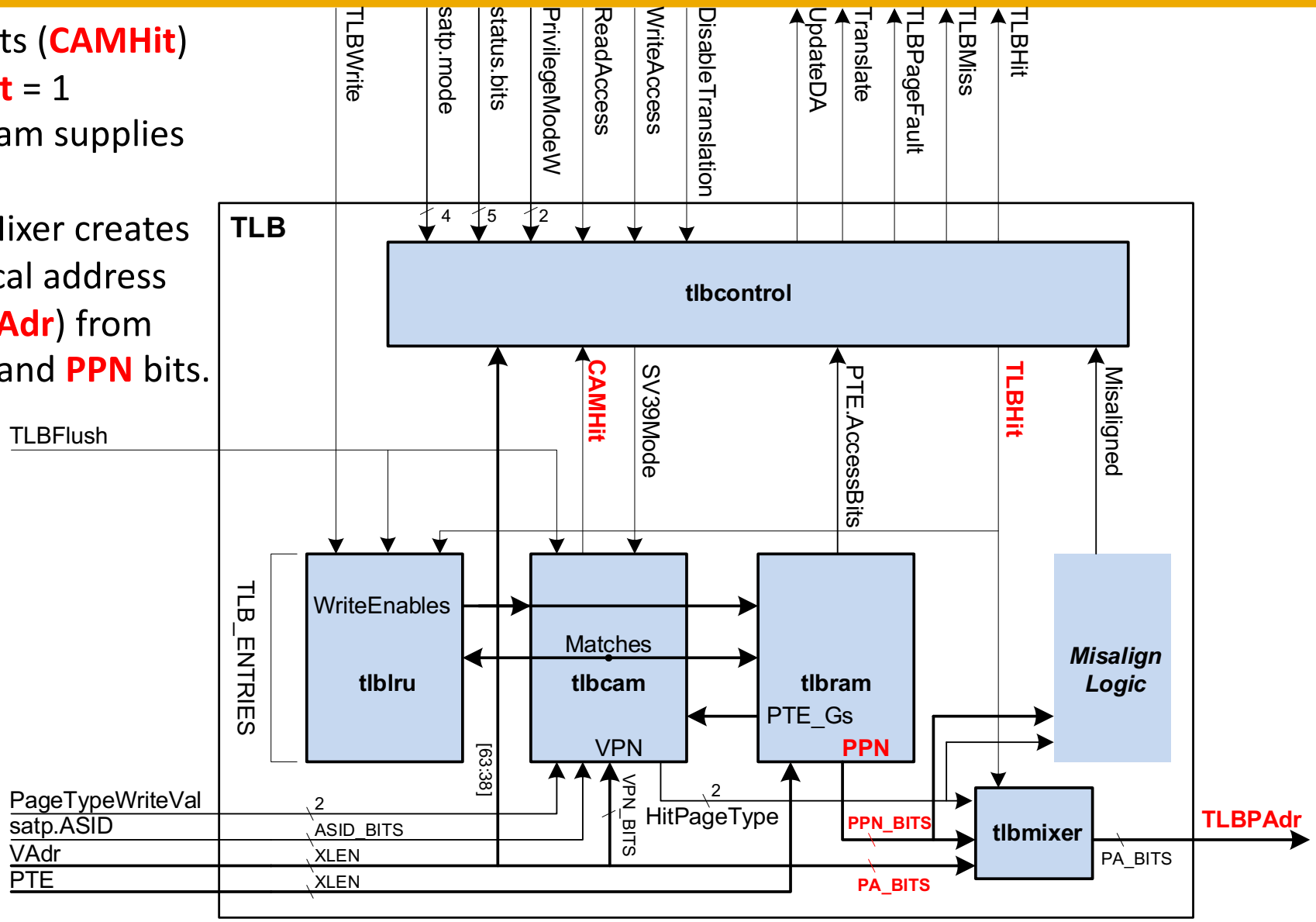
- Each TLB entry contains:
  - Tag
  - PTE (PPN + status bits)

Wally TLB

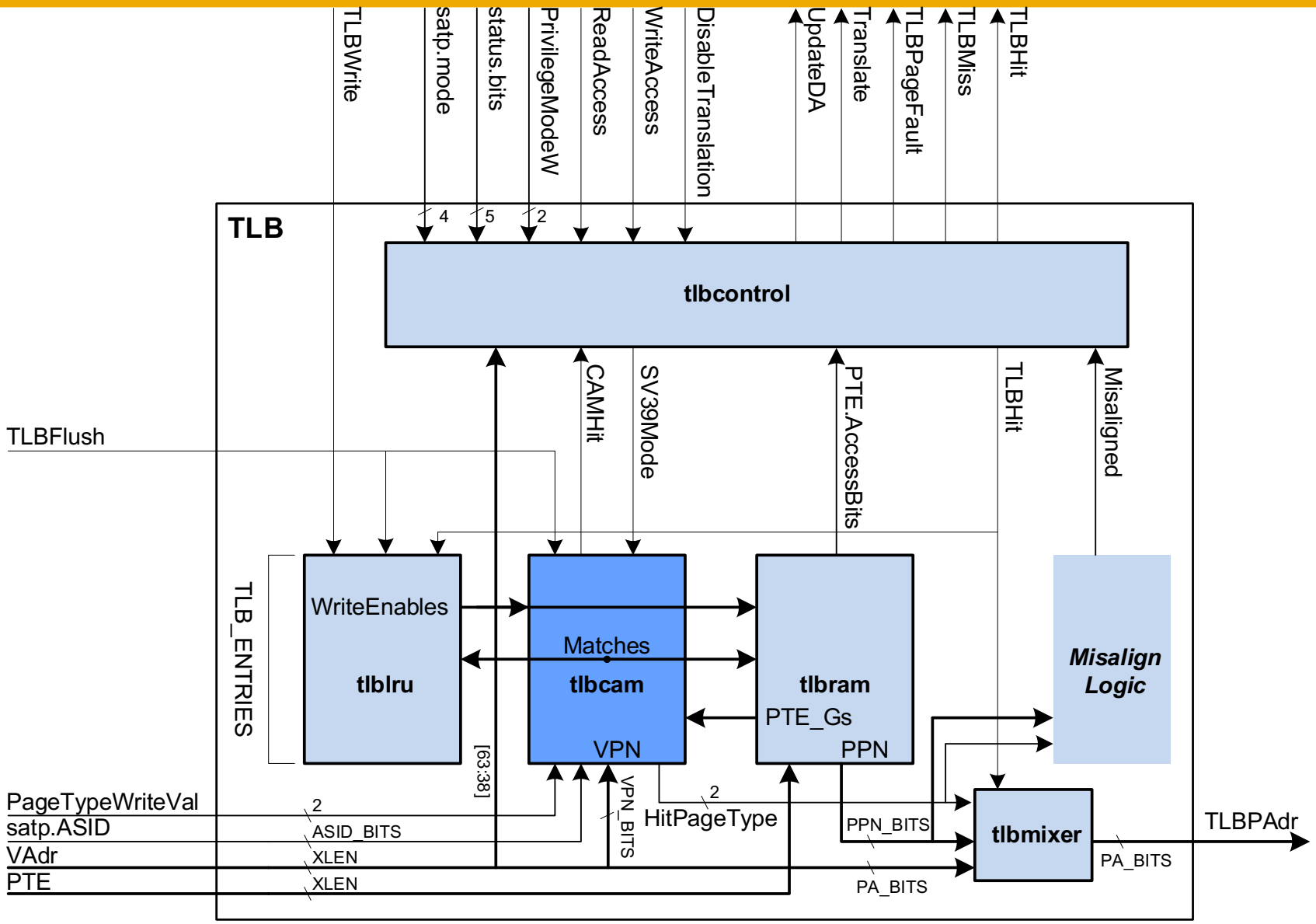

TLBWrite
satp.mode
status.bits
PrivilegeModeW
ReadAccess
WriteAccess
DisableTranslation
UpdateDA
Translate
TLBPageFault
TLBMiss
TLBHit
TLB
4
5
2
tlbcontrol
CAMHit
SV39Mode
PTE.AccessBits
TLBHit
Misaligned
TLBFlush
TLB_ENTRIES
WriteEnables
tlblru
Matches
tlbcam
VPN
tlbram
PTE_Gs
PPN
Misalign Logic
[63:38]
VPN_BITS
2
HitPageType
PPN_BITS
tlbmixer
TLBPAdr
PageTypeWriteVal
satp.ASID
VAdr
PTE
2
ASID_BITS
XLEN
XLEN
PA_BITS
PA_BITS

Wally TLB Hit

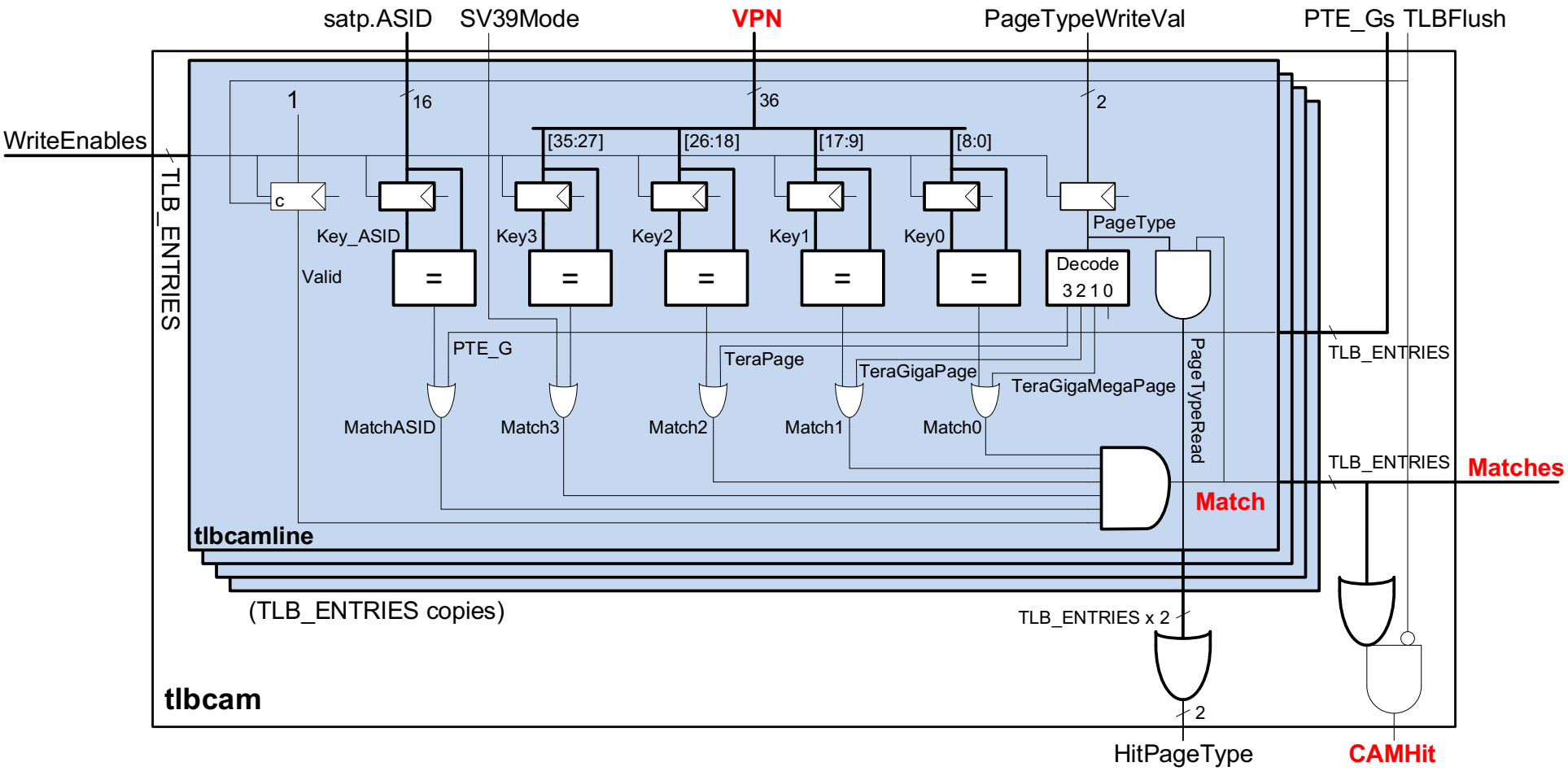
- Tag hits (**CAMHit**)
- **TLBHit** = 1
- TLB Ram supplies **PPN**
- TLB Mixer creates physical address (**TLBPAdr**) from **VAdr** and **PPN** bits.



Wally TLB CAM: Holds Tags

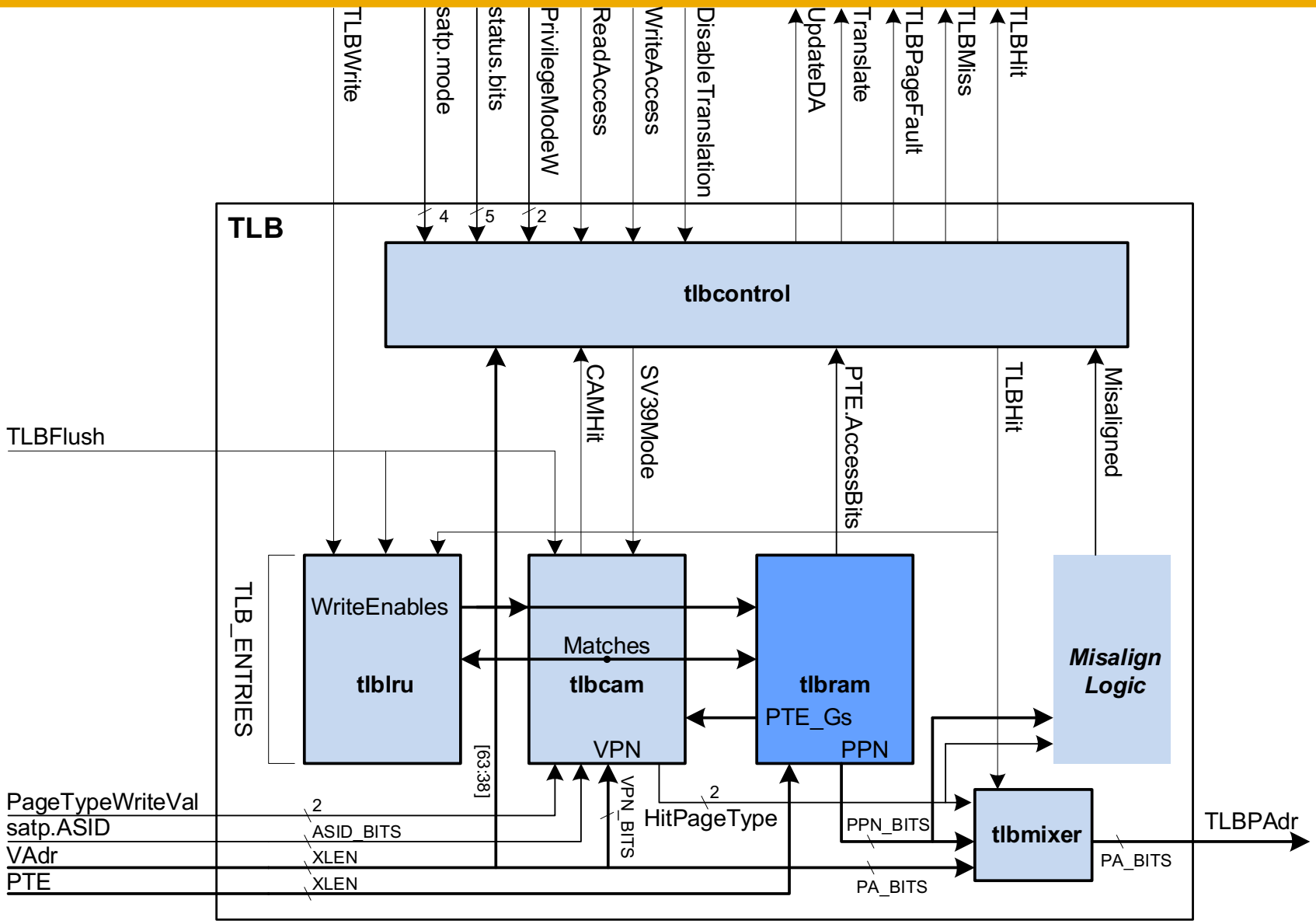


Wally TLB CAM: Holds Tags

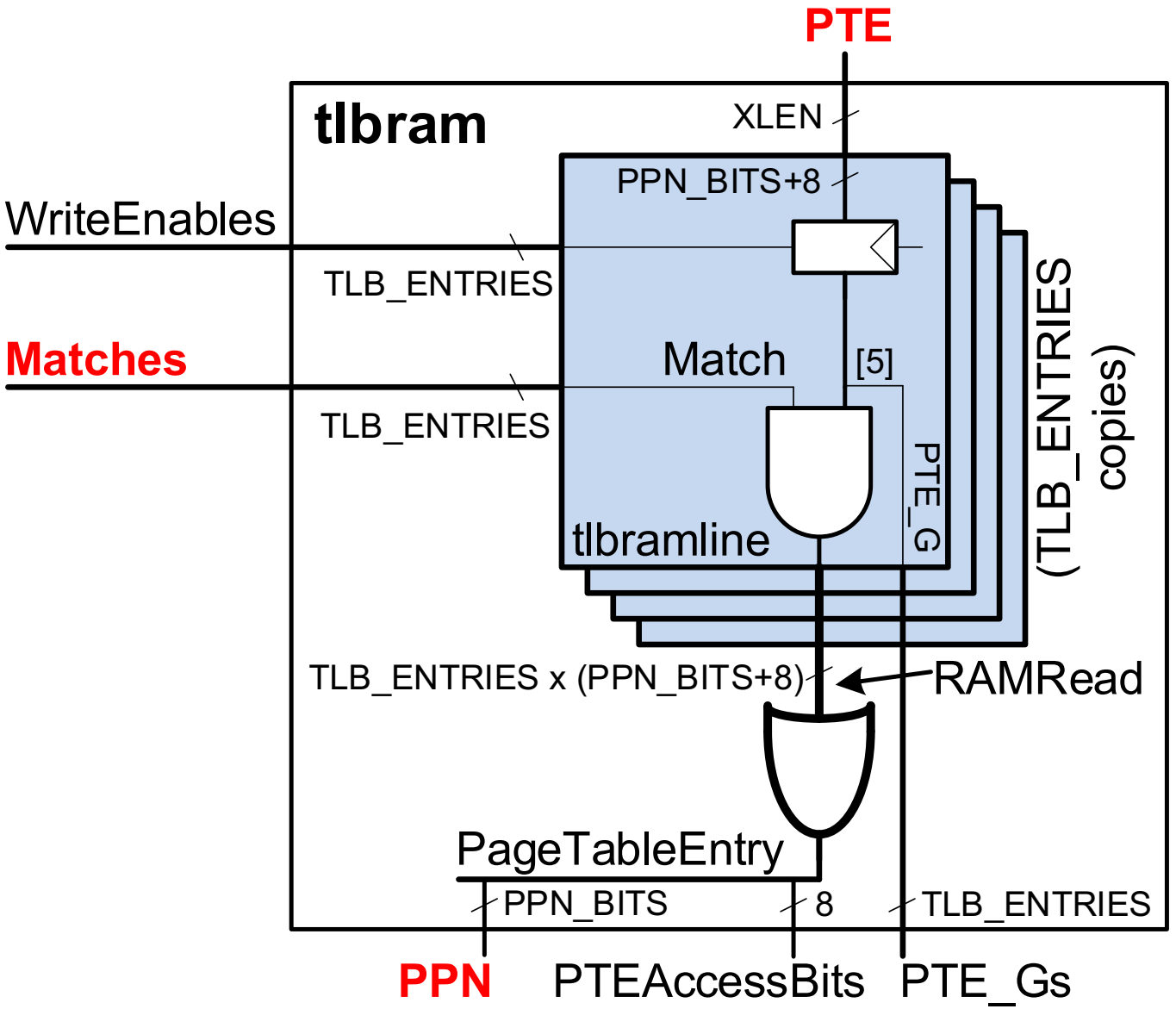


CAM: Content addressable memory

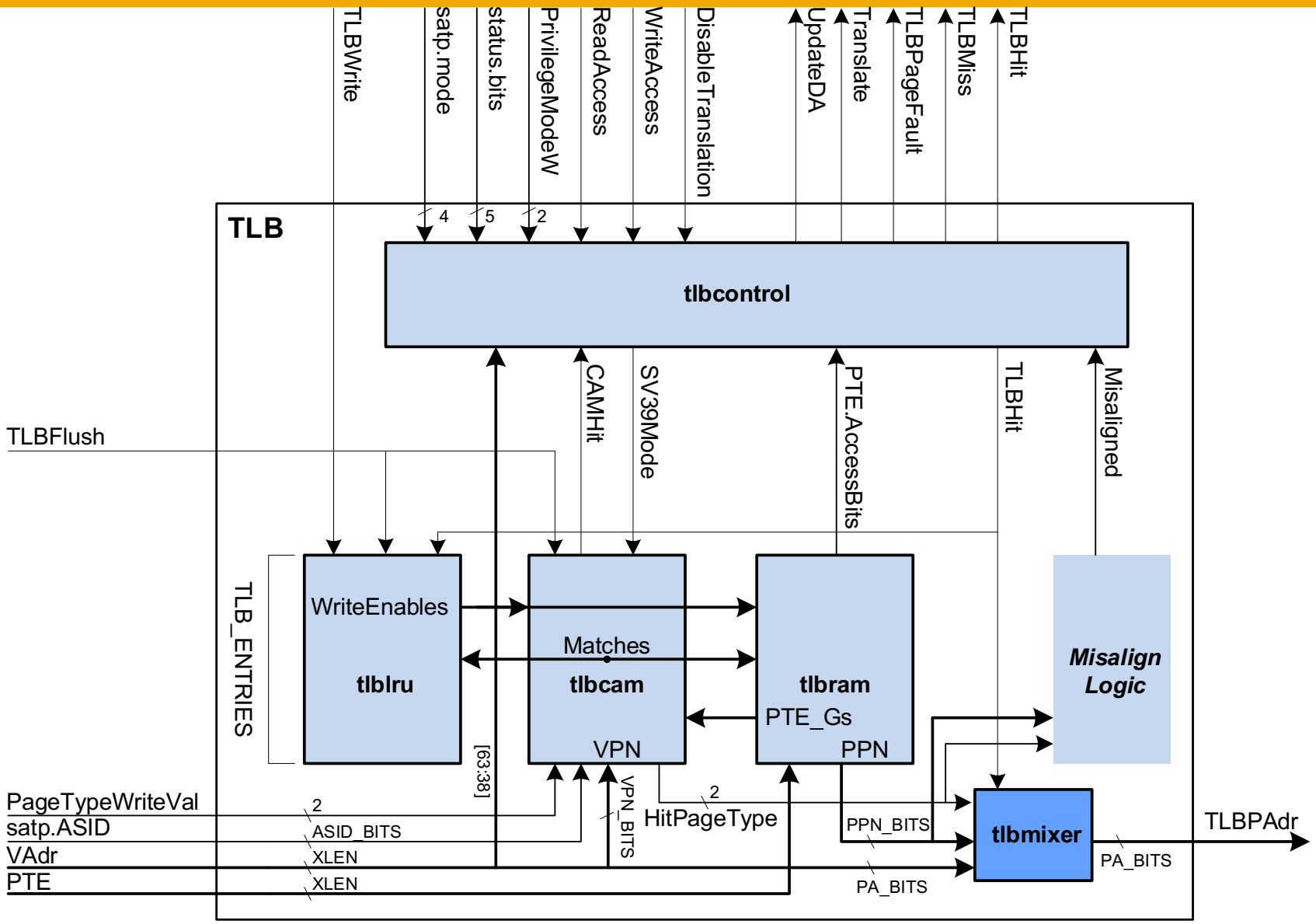
Wally TLB RAM: Holds PPN



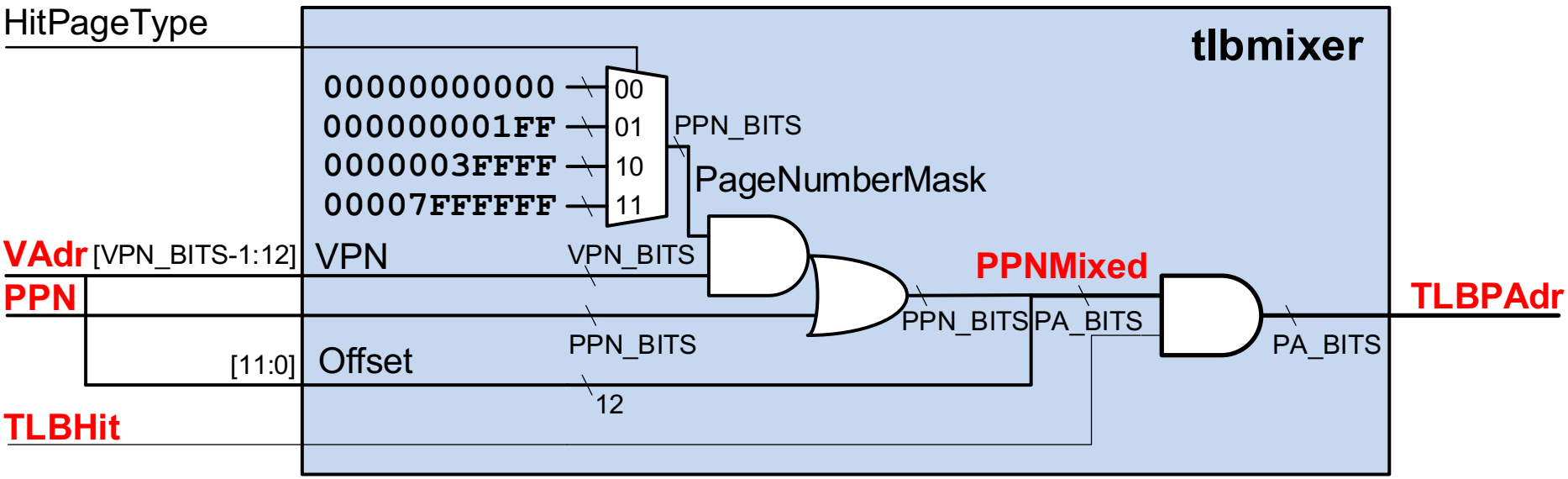
Wally TLB RAM: Holds PPN



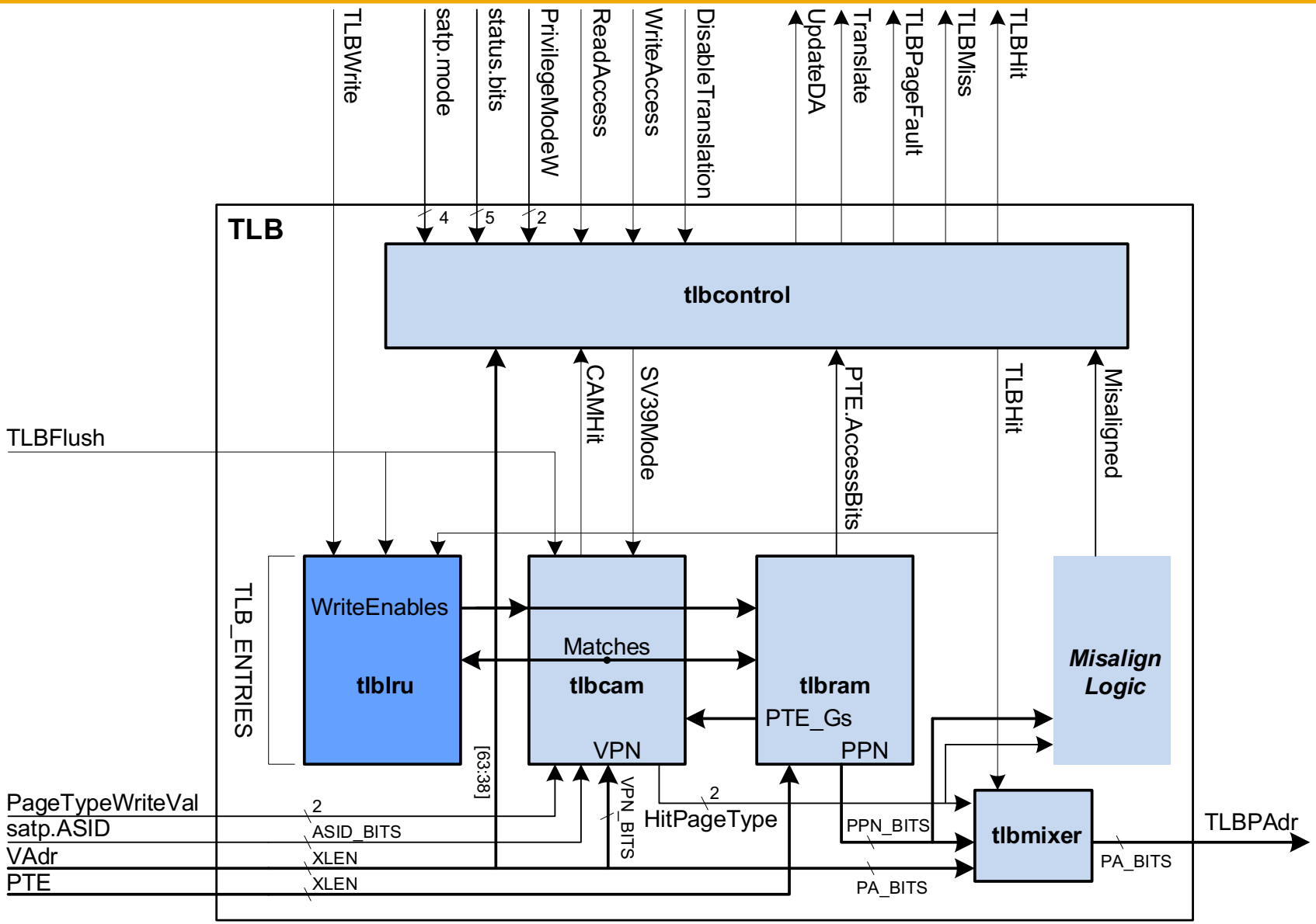
Wally TLB Mixer: PPN + Page Offset



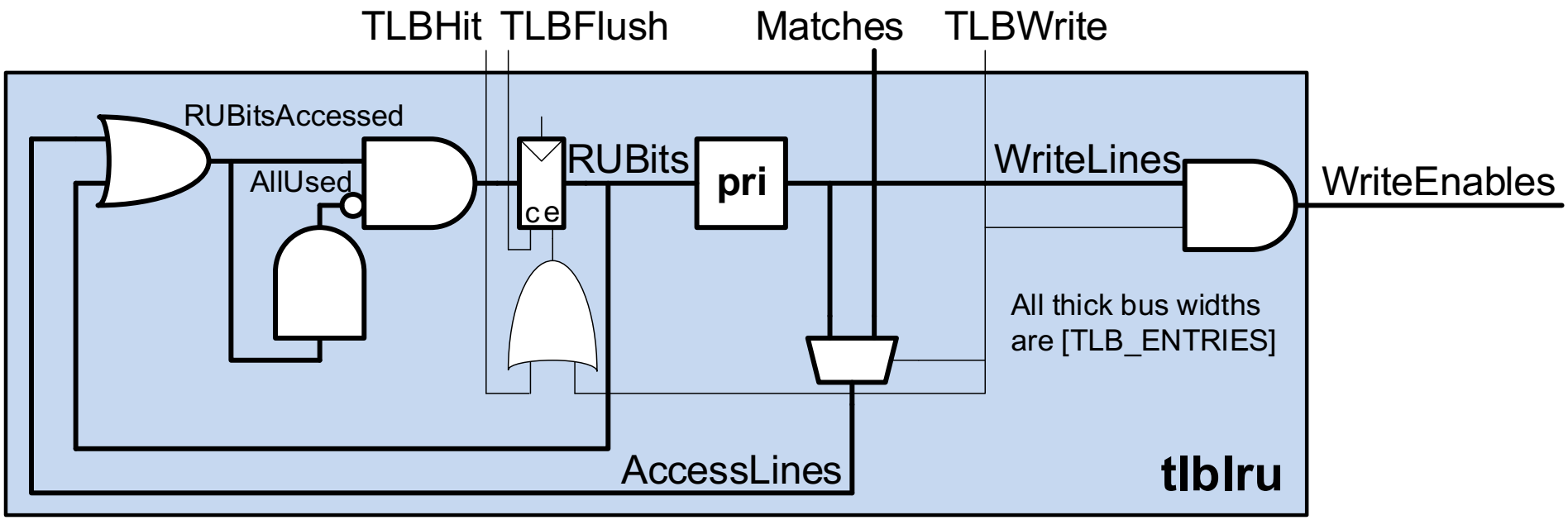
Wally TLB Mixer: PPN + Page Offset



Wally TLB LRU: Entry Replacement



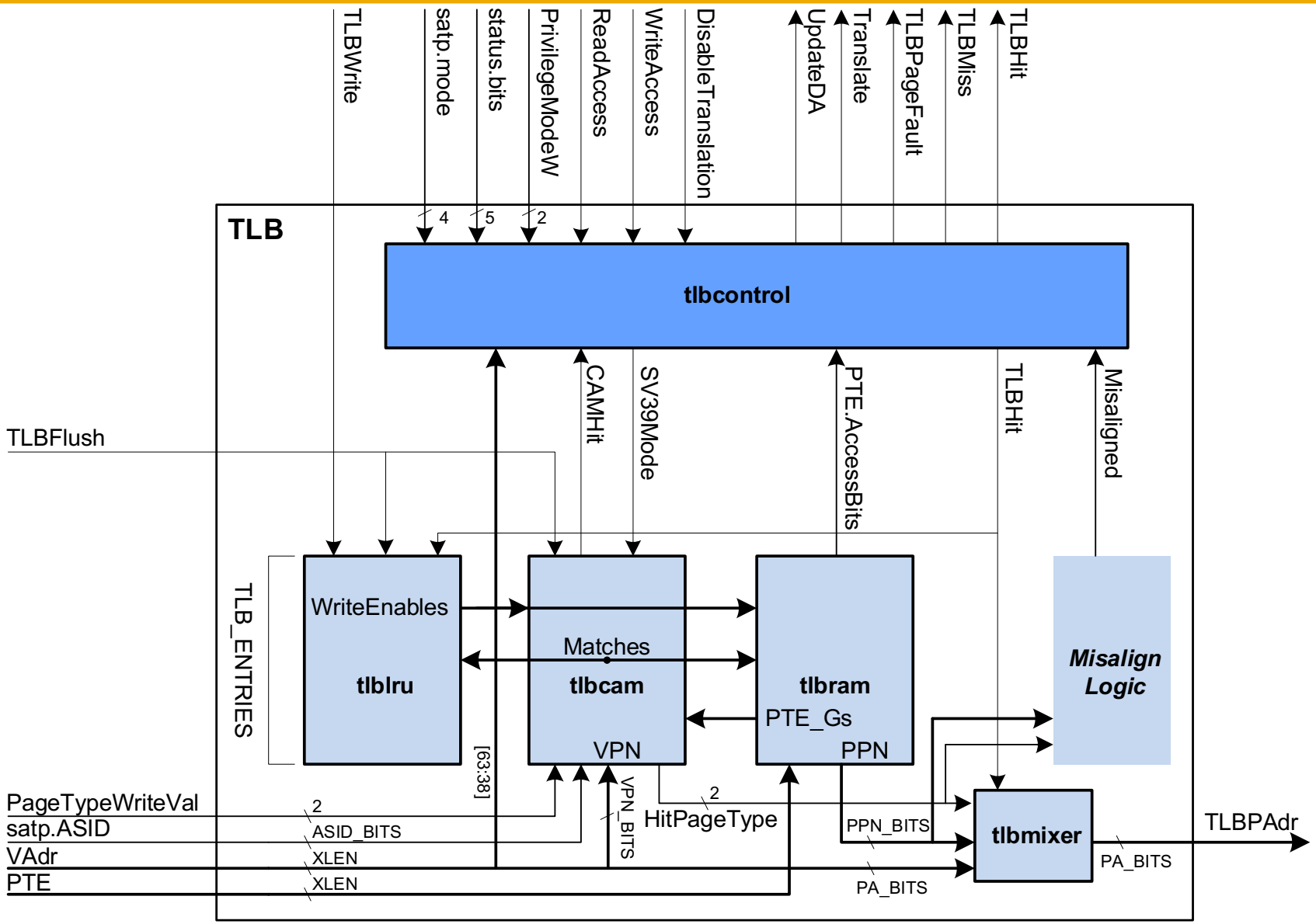
Wally TLB LRU



Approximate LRU

- Replaces first entry that hasn't been touched (read or written)
- If all have been touched, clears RU (recently used) bits (RUBits)

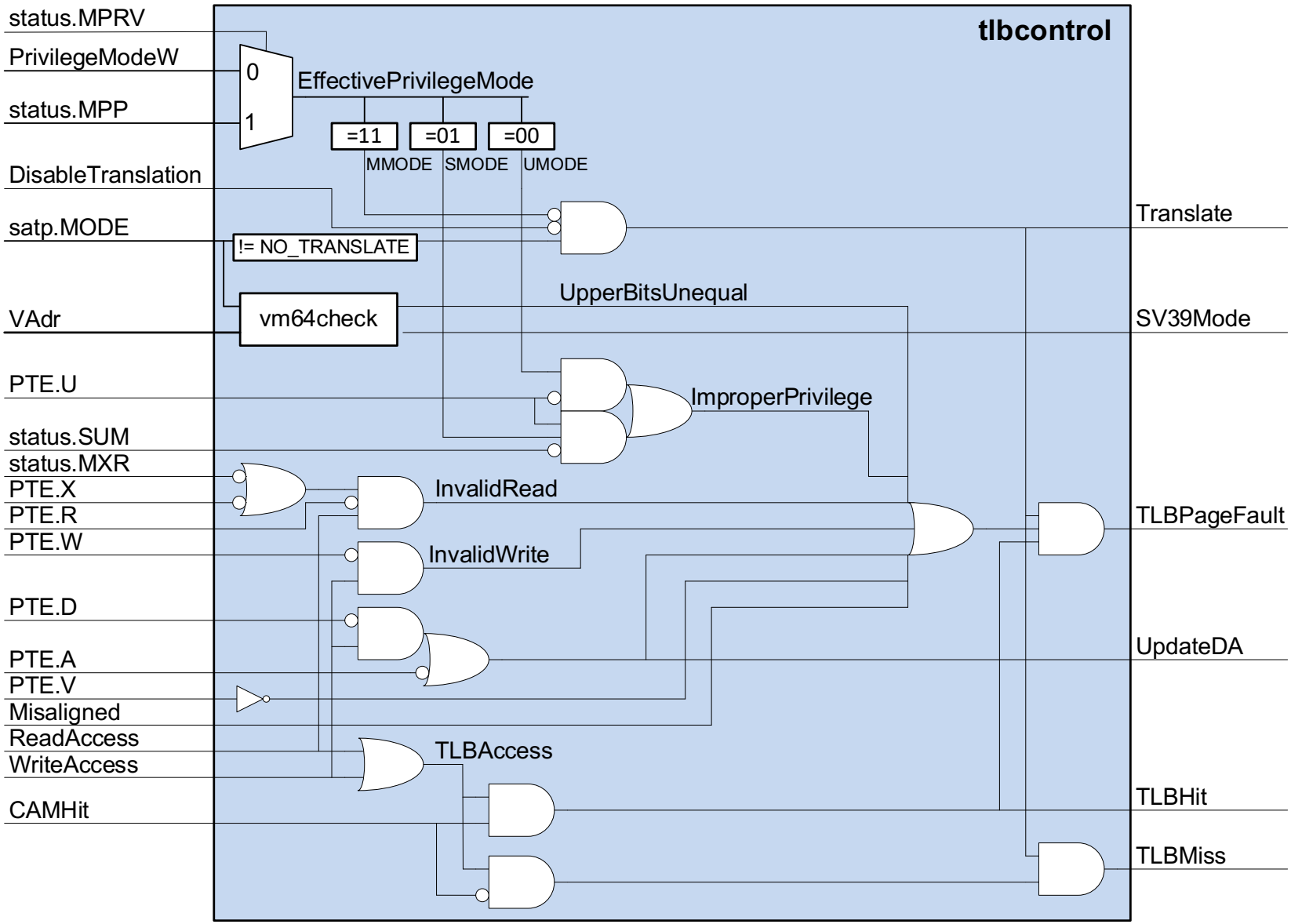
Wally TLB Control



TLB Control

- Detects hits and misses
- Generates faults depending on PTE bits (U, RWX, D, A, V) and status/satp bits

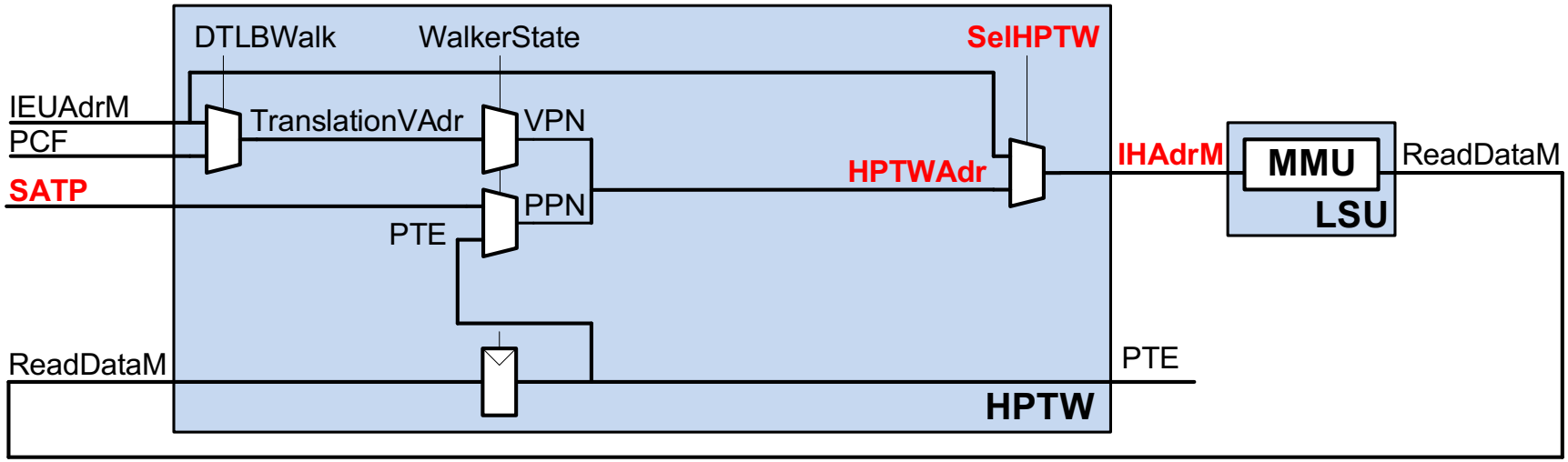
Wally TLB Control



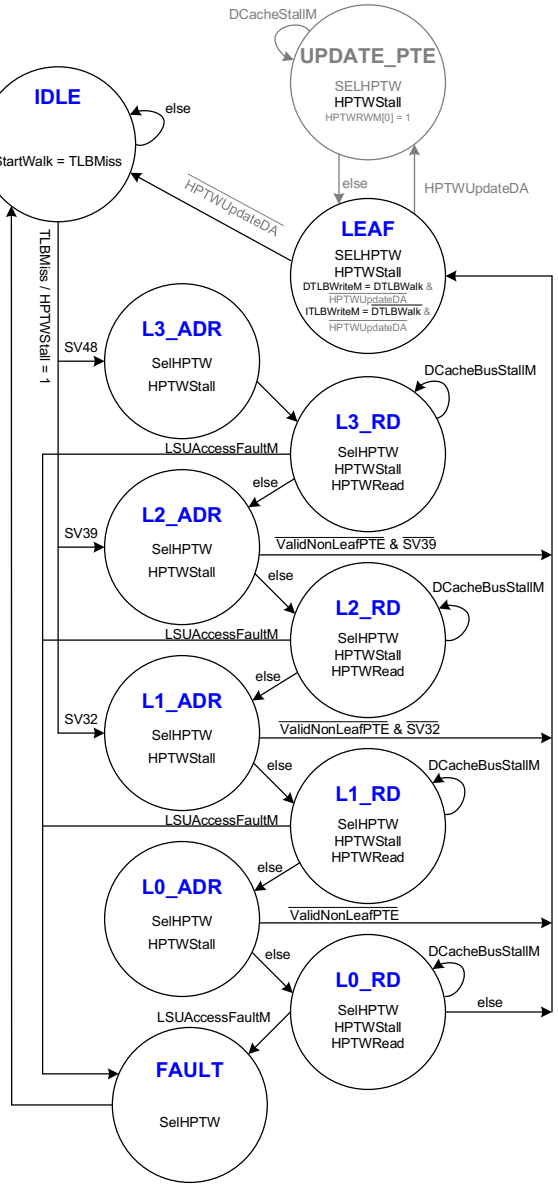
HPTW Datapath

Hardware Page Table Walker (HPTW):

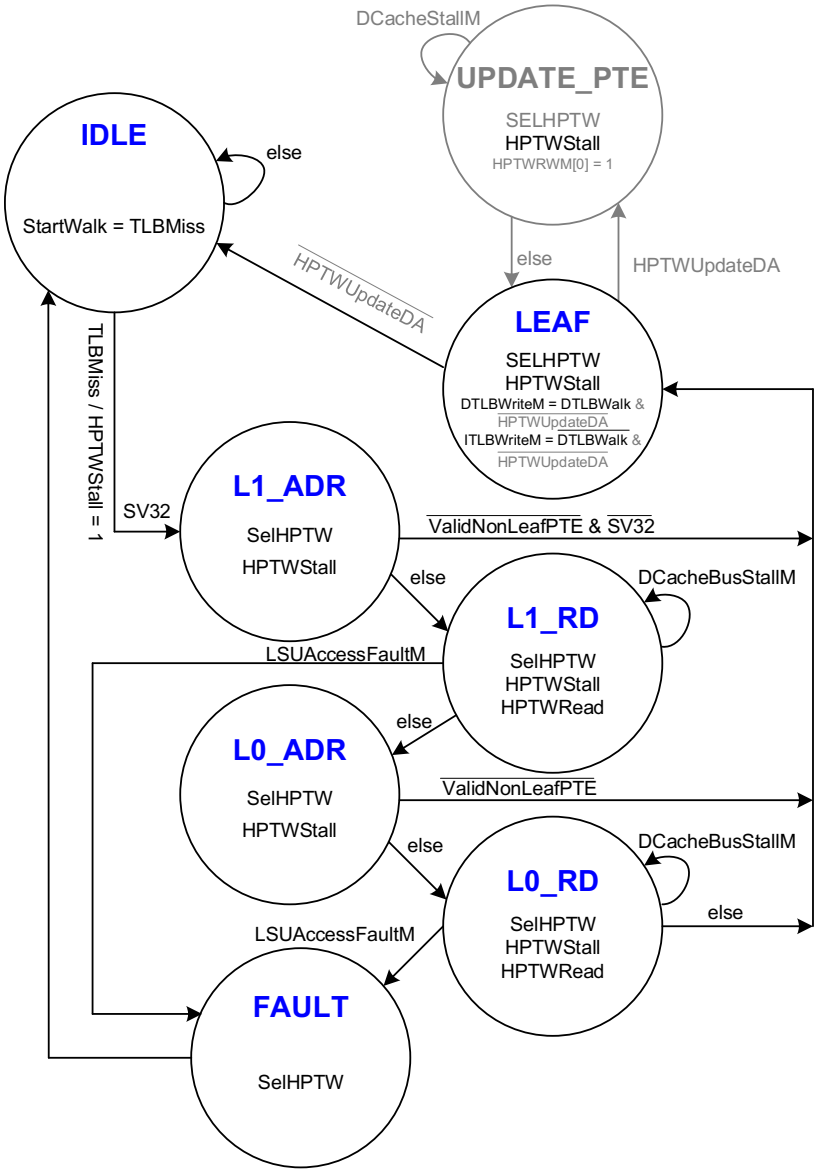
- When TLB misses
  - HPTW must traverse (walk) the page table levels to find the page
  - Instead of sending IEUAdrM (processor address), send address of page table HPTWAdr to MMU:
    - First access (upon miss) comes from `satp` register (SATP).
    - Lower-level page table accesses:
      - Upper bits: from PPN
      - Lower bits: from VPN



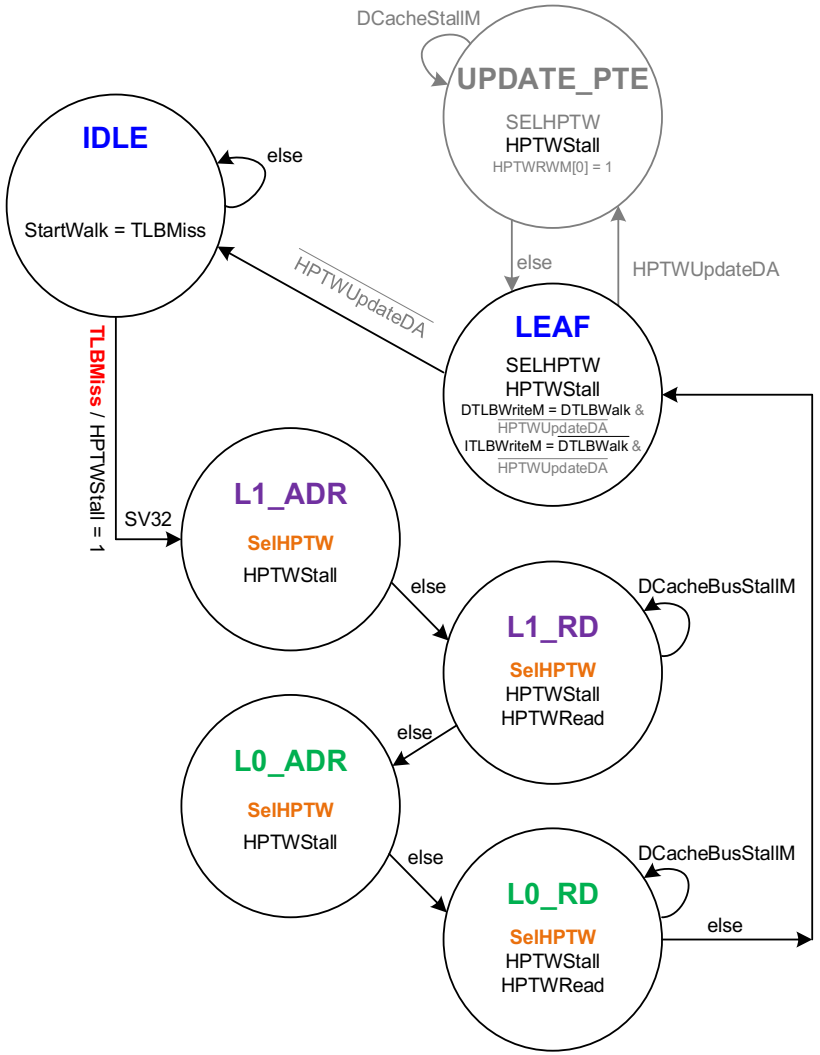
HPTW FSM


DCacheStallM
UPDATE_PTE
SELHPTW
HPTWStall
HPTWRWM[0] = 1
IDLE
else
StartWalk = TLBMiss
HPTWUpdateDA
else
HPTWUpdateDA
LEAF
SELHPTW
HPTWStall
DTLBWriteM = DTLBWalk &
HPTWUpdateDA
ITLBWriteM = DTLBWalk &
HPTWUpdateDA
TLBMiss / HPTWStall = 1
SV48
L3_ADR
SelHPTW
HPTWStall
DCacheBusStallM
L3_RD
SelHPTW
HPTWStall
HPTWRead
LSUAccessFaultM
else
SV39
L2_ADR
SelHPTW
HPTWStall
ValidNonLeafPTE & SV39
else
DCacheBusStallM
L2_RD
SelHPTW
HPTWStall
HPTWRead
LSUAccessFaultM
else
SV32
L1_ADR
SelHPTW
HPTWStall
ValidNonLeafPTE & SV32
else
DCacheBusStallM
L1_RD
SelHPTW
HPTWStall
HPTWRead
LSUAccessFaultM
else
L0_ADR
SelHPTW
HPTWStall
ValidNonLeafPTE
else
DCacheBusStallM
L0_RD
SelHPTW
HPTWStall
HPTWRead
else
LSUAccessFaultM
FAULT
SelHPTW

HPTW FSM: SV32 Only



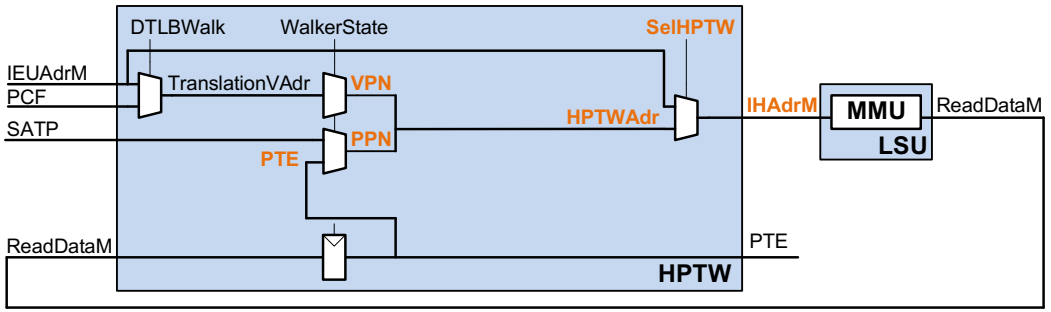
HPTW FSM: SV32 Simple TLB Miss



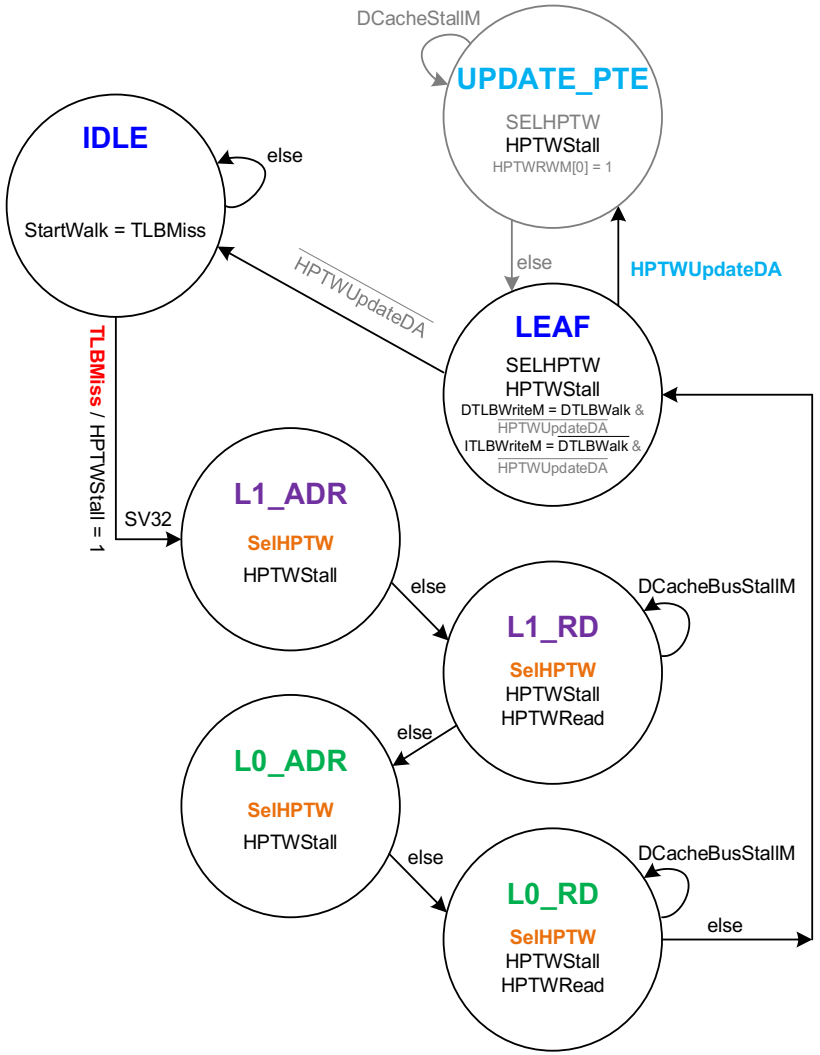
- **TLBMiss** = 1: Get missing translation

Get missing translation from page table via the D$

- **Level 1 Page Table Access:** Adr/Data (**L1_ADR/L1_RD**) to get PTE
- **Level 0 Page Table Access:** Adr/Data (**L0_ADR/L0_RD**) to get PTE = translation



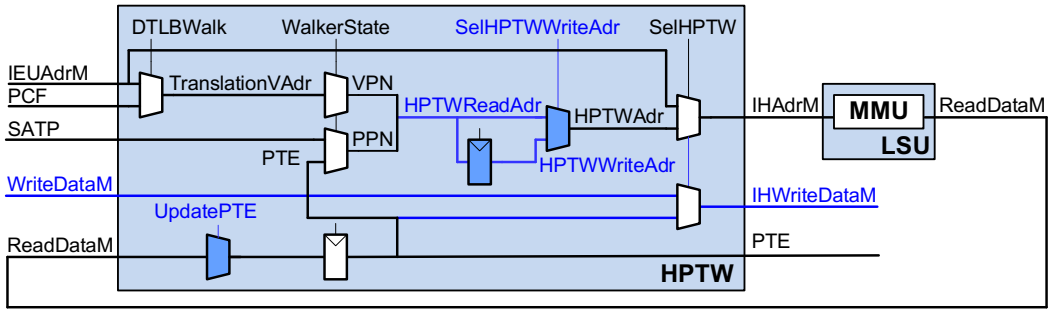
HPTW FSM: SV32 Simple TLB Miss



- **TLBMiss** = 1: Get missing translation

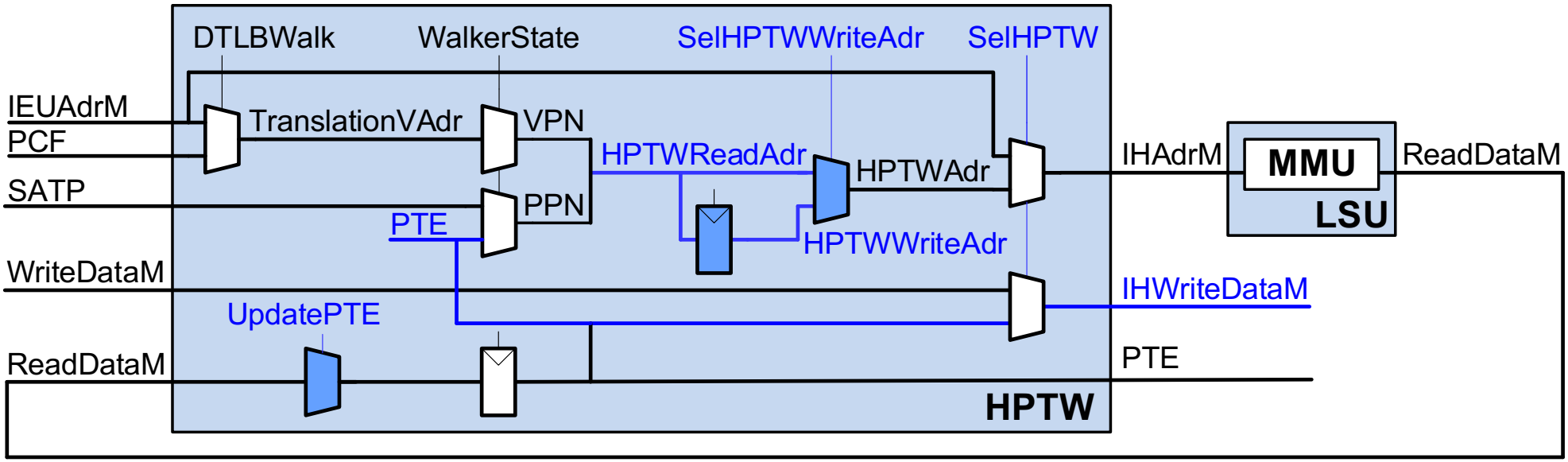
Get missing translation from page table via the D$

- **Level 1 Page Table Access:** Adr/Data (**L1_ADR/L1_RD**) to get PTE
- **Level 0 Page Table Access:** Adr/Data (**L0_ADR/L0_RD**) to get PTE = translation
- **Update PTE in Page Table (if necessary): UPDATE_PTE**



Updated HPTW datapath

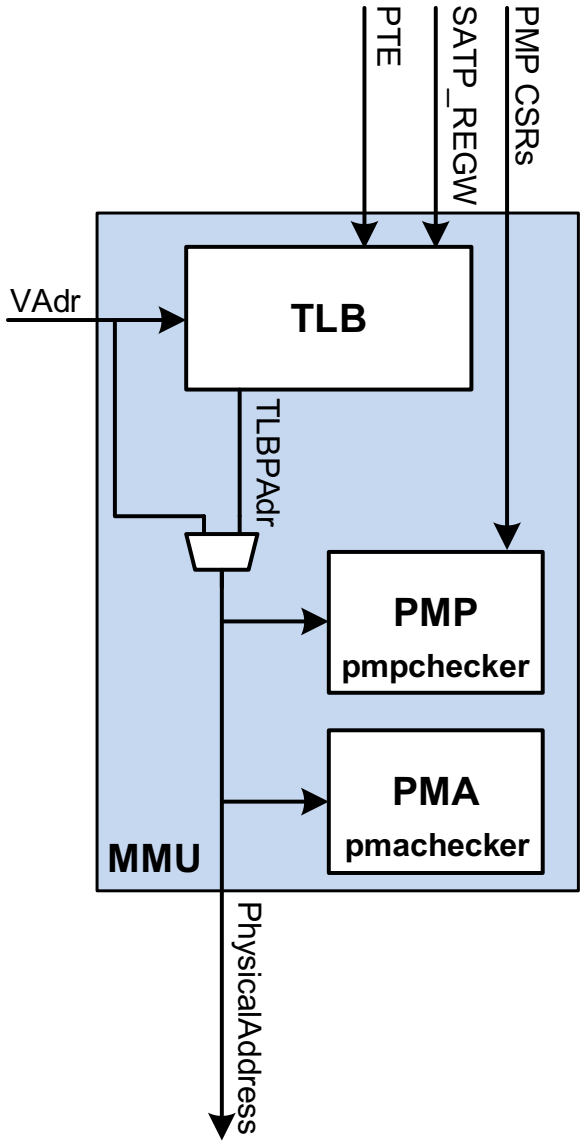
HPTW Datapath with PTE Update



Update PTE in page table (via the LSU's D$):

- **HPTW provides address and write data to LSU:** Address: HPTWWriteAdr = HPTWReadAdr delayed 1 cycle, Data: IHWriteDataM = PTE

Review: Wally MMU



MMU:

- **Produces physical address (`PhysicalAddress`):**
  - If virtual memory, **translates** virtual address (`VAdr`) to physical address
  - If no virtual memory, `PhysicalAddress` = `VAdr`
- **PMA** checks regions and **PMP** checks privilege and access type – they generate faults if violated
- **On TLB miss**, MMU sends a request to **HPTW to fetch missing PTE via the LSU's D$**

About these Notes

RISC-V System-on-Chip Design Lecture Notes

© 2025 D. Harris, J. Stine, R. Thompson, and S. Harris

These notes may be used and modified for educational and/or non-commercial purposes so long as the source is attributed.